

Securitatea Bazelor de Date

Oracle Database Security for Medical Appointments and Patient Records

Database: Oracle Database Enterprise Edition 19c

Development Environment: Oracle SQL Developer

Profesor coordonator: Letitia Marin

Student: Onoriu Elisabeta-maria

Grupa: 506

Program: Master, Anul II

Septembrie 2026

Cuprins

1. Introducere.....	4
1.1 Prezentarea modelului proiectat.....	4
1.2 Regulile modelului medical.....	5
1.3 Diagrama conceptuală.....	6
1.4 Schemele relaționale.....	7
1.5 Crearea tabelelor și inserarea datelor.....	11
1.6 Reguli de securitate aplicate asupra modelului.....	15
2. Criptarea datelor.....	16
2.1 Date sensibile în modelul medical.....	16
2.2 Implementarea criptării.....	16
2.3 Verificarea criptării.....	19
3. Auditarea activităților asupra bazei de date.....	21
3.1 Auditare standard.....	21
3.2 Trigger-i de auditare.....	22
3.3 Politici de auditare.....	23
3.4 Testarea auditării.....	23
4. Gestiunea utilizatorilor și a resurselor computaționale.....	24
4.1 Utilizatorii sistemului medical.....	25
4.2 Matrice proces-utilizator.....	27
4.3 Matrice entitate-proces.....	28
4.4 Matrice entitate-utilizator.....	29
4.5 Implementarea utilizatorilor, cotelor și profilelor.....	30
5. Privilegii și roluri.....	31
5.1 Roluri definite în sistem.....	32
5.2 Privilegii de sistem și privilegii pe obiecte.....	33
5.3 Ierarhia rolurilor.....	35
5.4 Testarea privilegiilor acordate.....	36
6. Aplicațiile pe baza de date și securitatea datelor.....	38
6.1 Contextul aplicației.....	39
6.2 SQL Injection.....	40
6.3 Procedură vulnerabilă.....	41

6.4 Procedură securizată.....	42
7. Mascarea datelor.....	43
7.1 Date mascate în modelul medical.....	43
7.2 Funcții de mascare.....	44
7.3 View-uri mascate.....	44
7.4 Testarea mascării.....	45
8. Concluzii.....	46

1. Introducere

Proiectul implementează un model de bază de date pentru gestionarea programărilor medicale și a evidenței pacienților, folosind Oracle Database 19c Enterprise Edition.

Scopul proiectului este construirea unui scenariu medical asupra căruia pot fi aplicate cerințele studiate în cadrul laboratorului de Securitatea Bazelor de Date. Modelul include informații despre pacienți, medici, specializări, programări, consultații, rezultate medicale, utilizatori ai bazei de date, roluri și activități auditate.

Baza de date este realizată local, iar scripturile SQL și PL/SQL sunt scrise și rulate în Oracle SQL Developer.

1.1. Prezentarea modelului proiectat

Modelul proiectat reprezintă un sistem medical simplificat, destinat unei clinici care gestionează pacienți, medici și programări. Sistemul permite înregistrarea pacienților, asocierea medicilor cu specializările lor, crearea programărilor medicale și salvarea informațiilor rezultate în urma consultațiilor.

Modelul gestionează următoarele elemente principale:

- Pacienți – persoane înregistrate în sistemul clinicii;
- Medici – personal medical care efectuează consultații;
- Specializări – domenii medicale asociate medicilor;
- Programări – întâlniri stabilite între pacienți și medici;
- Consultații – informații completate după desfășurarea unei programări;
- Rezultate medicale – rezultate sau observații medicale asociate pacienților;
- Utilizatori ai bazei de date – conturi folosite pentru accesarea sistemului;
- Roluri – categorii de acces definite pentru utilizatori;
- Privilegii – drepturi asociate rolurilor;
- Jurnal de audit – tabel utilizat pentru înregistrarea unor activități din sistem.

Modelul este construit astfel încât să permită aplicarea cerințelor de securitate asupra unei baze de date medicale, păstrând o structură clară și ușor de testat.

1.2 Regulile modelului medical

Modelul medical respectă următoarele reguli de business:

1. Un pacient poate avea una sau mai multe programări medicale.
2. Fiecare programare este asociată unui singur pacient și unui singur medic.
3. Un medic poate avea una sau mai multe specializări medicale.
4. O specializare medicală poate fi asociată mai multor medici.
5. Relația dintre medici și specializări este gestionată printr-o tabelă intermediară.
6. O programare poate avea unul dintre următoarele statusuri: SCHEDULED, IN_PROGRESS, COMPLETED sau CANCELLED.
7. O consultație poate avea o programare asociată, însă asocierea nu este obligatorie pentru pacienții primiți fără programare.
8. Fiecare consultație este asociată unei singure programări.
9. Un pacient poate avea mai multe rezultate medicale.
10. Fiecare rezultat medical este asociat unui pacient și unui medic.
11. Utilizatorii bazei de date sunt diferențiați prin roluri, iar fiecare rol are drepturi specifice asupra datelor și operațiilor.

1.3 Diagrama conceptuala

Diagrama conceptuală de mai jos prezintă entitățile principale ale modelului medical și relațiile dintre acestea, evidențiind atât componenta funcțională a clinicii, cât și componenta de administrare a accesului în baza de date.

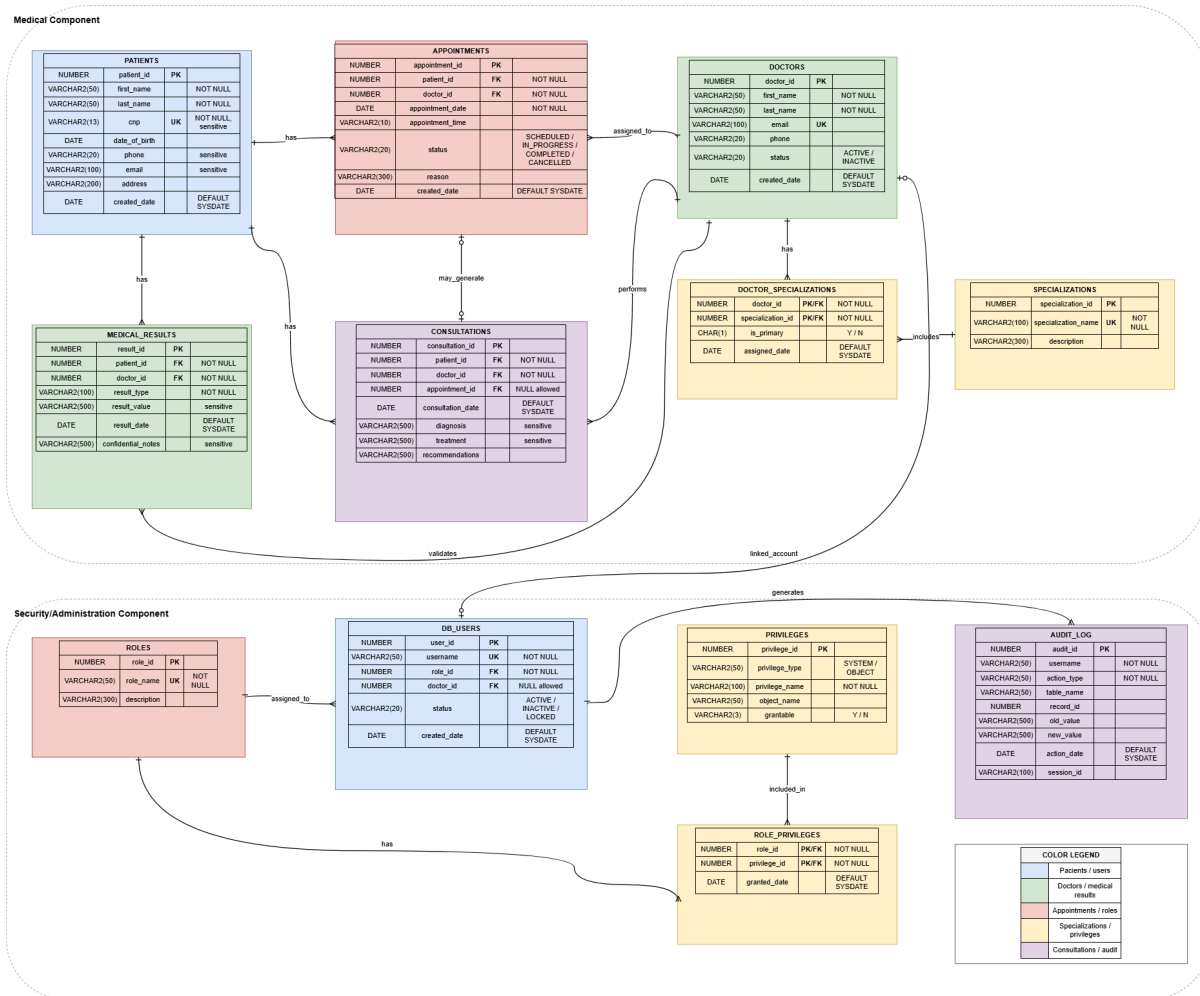


Figura 1. Diagramă Entitate-Relație (ERD)

Entitățile principale ale modelului sunt legate prin relații de tip one-to-many sau prin tabele intermediare, acolo unde este necesară reprezentarea unei relații de tip many-to-many.

Entități principale:

- PATIENTS (1) $\longleftrightarrow$ (N) APPOINTMENTS
- DOCTORS (1) $\longleftrightarrow$ (N) APPOINTMENTS
- PATIENTS (1) $\longleftrightarrow$ (N) CONSULTATIONS
- DOCTORS (1) $\longleftrightarrow$ (N) CONSULTATIONS
- APPOINTMENTS (0..1) $\longleftrightarrow$ (0..1) CONSULTATIONS
- PATIENTS (1) $\longleftrightarrow$ (N) MEDICAL_RESULTS
- DOCTORS (1) $\longleftrightarrow$ (N) MEDICAL_RESULTS
- DOCTORS (1) $\longleftrightarrow$ (N) DOCTOR_SPECIALIZATIONS
- SPECIALIZATIONS (1) $\longleftrightarrow$ (N) DOCTOR_SPECIALIZATIONS
- ROLES (1) $\longleftrightarrow$ (N) DB_USERS
- ROLES (1) $\longleftrightarrow$ (N) ROLE_PRIVILEGES
- PRIVILEGES (1) $\longleftrightarrow$ (N) ROLE_PRIVILEGES
- DB_USERS (1) $\longleftrightarrow$ (N) AUDIT_LOG

1.4 Schemele Relaționale

Figura 2. Schema tabelului patients

```
CREATE TABLE patients (  
    patient_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50) NOT NULL,  
    last_name VARCHAR2(50) NOT NULL,  
    cnp VARCHAR2(13) NOT NULL UNIQUE,  
    date_of_birth DATE,  
    phone VARCHAR2(20),  
    email VARCHAR2(100),  
    address VARCHAR2(200),  
    created_date DATE DEFAULT SYSDATE  
);
```

Figura 3. Schema tabelului doctors

```
CREATE TABLE doctors (  
    doctor_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50) NOT NULL,  
    last_name VARCHAR2(50) NOT NULL,  
    email VARCHAR2(100) UNIQUE,  
    phone VARCHAR2(20),  
    status VARCHAR2(20) DEFAULT 'ACTIVE',  
    created_date DATE DEFAULT SYSDATE,  
    CONSTRAINT chk_doctors_status  
        CHECK (status IN ('ACTIVE', 'INACTIVE'))  
);
```

Figura 4. Schema tabelului specializations

```
CREATE TABLE specializations (  
    specialization_id NUMBER PRIMARY KEY,  
    specialization_name VARCHAR2(100) NOT NULL UNIQUE,  
    description VARCHAR2(300)  
);
```

Figura 5. Schema tabelului doctor_specializations

```
CREATE TABLE doctor_specializations (  
    doctor_id NUMBER NOT NULL,  
    specialization_id NUMBER NOT NULL,  
    is_primary CHAR(1) DEFAULT 'N',  
    assigned_date DATE DEFAULT SYSDATE,  
  
    CONSTRAINT pk_doctor_specializations  
        PRIMARY KEY (doctor_id, specialization_id),  
  
    CONSTRAINT fk_ds_doctor  
        FOREIGN KEY (doctor_id)  
        REFERENCES doctors(doctor_id),  
  
    CONSTRAINT fk_ds_specialization  
        FOREIGN KEY (specialization_id)  
        REFERENCES specializations(specialization_id),  
  
    CONSTRAINT chk_ds_primary  
        CHECK (is_primary IN ('Y', 'N'))  
);
```


Figura 6. Schema tabelului appointments

```
CREATE TABLE appointments (  
    appointment_id NUMBER PRIMARY KEY,  
    patient_id NUMBER NOT NULL,  
    doctor_id NUMBER NOT NULL,  
    appointment_date DATE NOT NULL,  
    appointment_time VARCHAR2(10),  
    status VARCHAR2(20) DEFAULT 'SCHEDULED',  
    reason VARCHAR2(300),  
    created_date DATE DEFAULT SYSDATE,  
  
    CONSTRAINT fk_appointments_patient  
        FOREIGN KEY (patient_id)  
        REFERENCES patients(patient_id),  
  
    CONSTRAINT fk_appointments_doctor  
        FOREIGN KEY (doctor_id)  
        REFERENCES doctors(doctor_id),  
  
    CONSTRAINT chk_appointments_status  
        CHECK (status IN ('SCHEDULED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'))  
);
```

Figura 7. Schema tabelului consultations

```
CREATE TABLE consultations (  
    consultation_id NUMBER PRIMARY KEY,  
    patient_id NUMBER NOT NULL,  
    doctor_id NUMBER NOT NULL,  
    appointment_id NUMBER,  
    consultation_date DATE DEFAULT SYSDATE,  
    diagnosis VARCHAR2(500),  
    treatment VARCHAR2(500),  
    recommendations VARCHAR2(500),  
  
    CONSTRAINT fk_consultations_patient  
        FOREIGN KEY (patient_id)  
        REFERENCES patients(patient_id),  
  
    CONSTRAINT fk_consultations_doctor  
        FOREIGN KEY (doctor_id)  
        REFERENCES doctors(doctor_id),  
  
    CONSTRAINT fk_consultations_appointment  
        FOREIGN KEY (appointment_id)  
        REFERENCES appointments(appointment_id)  
);
```

Figura 8. Schema tabelului Medical Results

```
CREATE TABLE medical_results (  
    result_id NUMBER PRIMARY KEY,  
    patient_id NUMBER NOT NULL,  
    doctor_id NUMBER NOT NULL,  
    result_type VARCHAR2(100) NOT NULL,  
    result_value VARCHAR2(500),  
    result_date DATE DEFAULT SYSDATE,  
    confidential_notes VARCHAR2(500),  
  
    CONSTRAINT fk_results_patient  
        FOREIGN KEY (patient_id)  
        REFERENCES patients(patient_id),  
  
    CONSTRAINT fk_results_doctor  
        FOREIGN KEY (doctor_id)  
        REFERENCES doctors(doctor_id)  
);
```

1.5 Crearea tabelelor și inserarea datelor

Baza de date conține 12 tabele create pentru modelarea sistemului medical și pentru implementarea mecanismelor de securitate asociate. Tabelele acoperă atât partea operațională a sistemului, prin evidența pacienților, medicilor, programărilor, consultațiilor și rezultatelor medicale, cât și partea de administrare, prin utilizatori, roluri, privilegii și jurnalul de audit.

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'MED_APP_ADMIN' database selected. Below it, the 'Reports' pane lists various report categories. The main workspace is divided into two panes: 'Worksheet' and 'Query Builder'. The 'Worksheet' pane contains the following SQL query:

```
SELECT table_name
FROM user_tables
ORDER BY table_name;
```

The 'Query Result' pane at the bottom right shows the results of the query, listing 12 tables in the MED_APP_ADMIN database:

TABLE_NAME
1 APPOINTMENTS
2 AUDIT_LOG
3 CONSULTATIONS
4 DB_USERS
5 DOCTORS
6 DOCTOR_SPECIALIZATIONS
7 MEDICAL_RESULTS
8 PATIENTS
9 PRIVILEGES
10 ROLES
11 ROLE_PRIVILEGES
12 SPECIALIZATIONS

Figura 9. Confirmarea creării tabelelor modelului medical

Constrângeri și reguli implementate:

- PRIMARY KEY pentru identificarea unică a înregistrărilor din fiecare tabel;
- FOREIGN KEY pentru menținerea relațiilor dintre pacienți, medici, programări, consultații și rezultate medicale;
- UNIQUE pentru câmpuri care trebuie să aibă valori distincte, precum CNP-ul pacientului, emailul medicului, username-ul utilizatorului sau numele rolului;
- CHECK pentru validarea valorilor permise, precum statusul programărilor, statusul utilizatorilor sau tipul privilegiilor;
- valori DEFAULT pentru completarea automată a unor câmpuri, precum datele de creare sau statusurile inițiale.

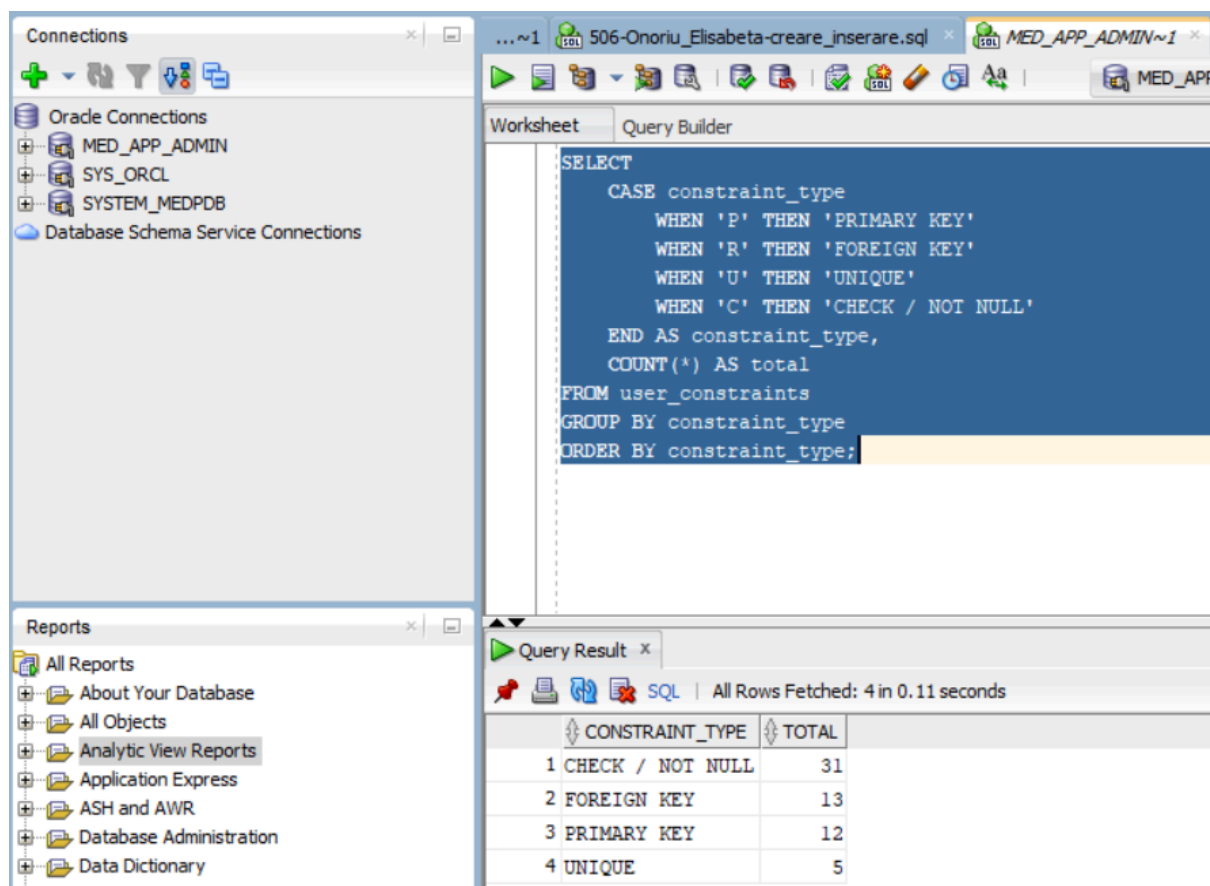
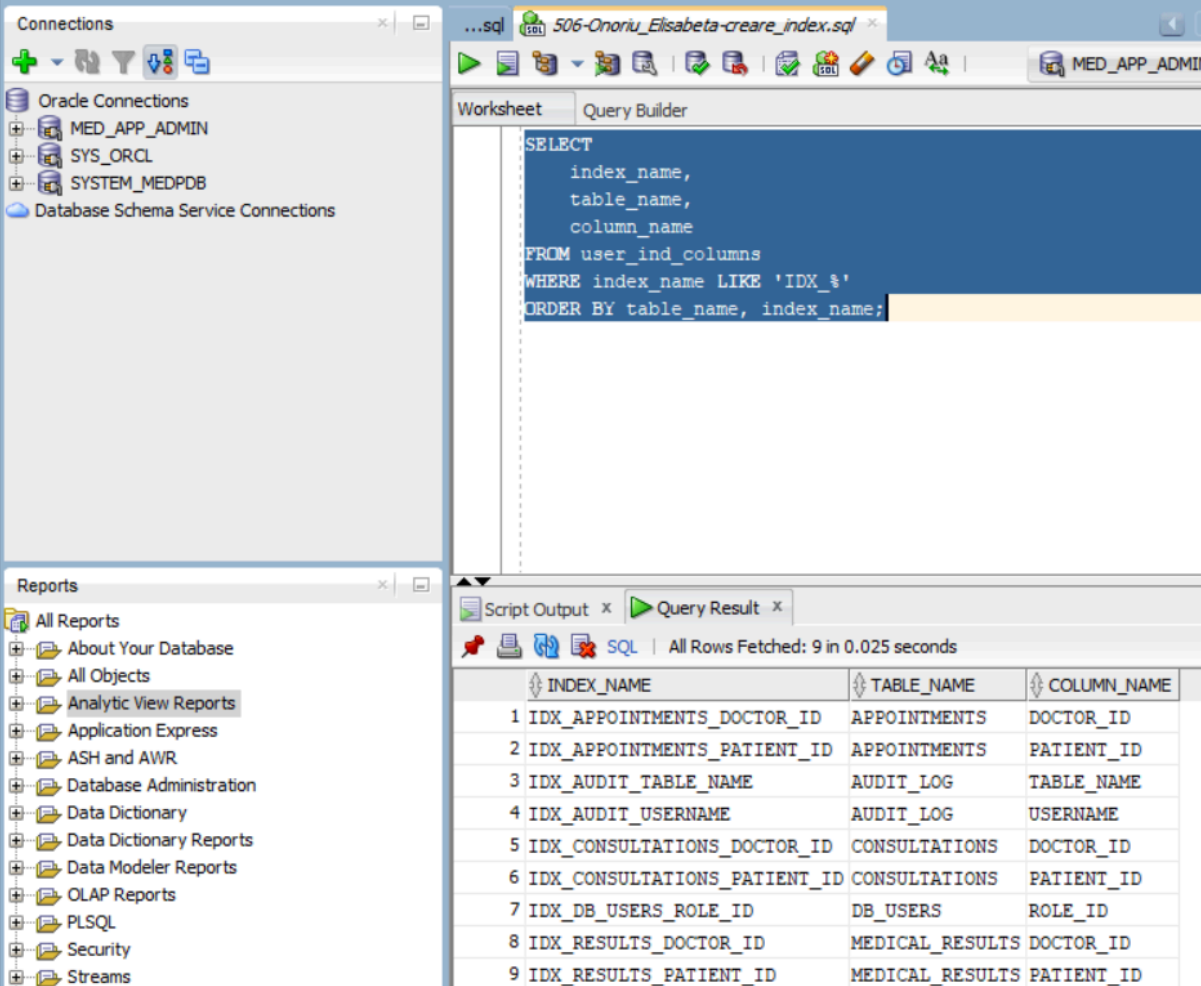


Figura 10. Constraints

Indexuri pentru performanță:

- IDX_APPOINTMENTS_PATIENT_ID — APPOINTMENTS(patient_id)
- IDX_APPOINTMENTS_DOCTOR_ID — APPOINTMENTS(doctor_id)
- IDX_CONSULTATIONS_PATIENT_ID — CONSULTATIONS(patient_id)
- IDX_CONSULTATIONS_DOCTOR_ID — CONSULTATIONS(doctor_id)
- IDX_RESULTS_PATIENT_ID — MEDICAL_RESULTS(patient_id)
- IDX_RESULTS_DOCTOR_ID — MEDICAL_RESULTS(doctor_id)
- IDX_DB_USERS_ROLE_ID — DB_USERS(role_id)
- IDX_AUDIT_USERNAME — AUDIT_LOG(username)
- IDX_AUDIT_TABLE_NAME — AUDIT_LOG(table_name)



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows 'MED_APP_ADMIN' selected. The main window shows a SQL query in the 'Query Builder' tab:

```
SELECT
    index_name,
    table_name,
    column_name
FROM user_ind_columns
WHERE index_name LIKE 'IDX_%'
ORDER BY table_name, index_name;
```

Below the query, the 'Query Result' pane shows the results of the query, with 9 rows fetched in 0.025 seconds. The results are as follows:

	INDEX_NAME	TABLE_NAME	COLUMN_NAME
1	IDX_APPOINTMENTS_DOCTOR_ID	APPOINTMENTS	DOCTOR_ID
2	IDX_APPOINTMENTS_PATIENT_ID	APPOINTMENTS	PATIENT_ID
3	IDX_AUDIT_TABLE_NAME	AUDIT_LOG	TABLE_NAME
4	IDX_AUDIT_USERNAME	AUDIT_LOG	USERNAME
5	IDX_CONSULTATIONS_DOCTOR_ID	CONSULTATIONS	DOCTOR_ID
6	IDX_CONSULTATIONS_PATIENT_ID	CONSULTATIONS	PATIENT_ID
7	IDX_DB_USERS_ROLE_ID	DB_USERS	ROLE_ID
8	IDX_RESULTS_DOCTOR_ID	MEDICAL_RESULTS	DOCTOR_ID
9	IDX_RESULTS_PATIENT_ID	MEDICAL_RESULTS	PATIENT_ID

Figura 11. Indexuri de performanta

Date de test inserate:

- 3 pacienți înregistrați în sistem;
- 3 medici activi;
- 4 specializări medicale;
- 4 asocieri între medici și specializări;
- 3 programări medicale;
- 2 consultații, dintre care una fără programare asociată;
- 2 rezultate medicale;
- 6 roluri definite pentru controlul accesului;
- 4 utilizatori ai bazei de date;
- 5 privilegii definite;
- 6 asocieri între roluri și privilegii.

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows 'MED_APP_ADMIN' selected. The 'Reports' pane lists various report categories. The main window shows a SQL query in the 'Query Builder' tab, which is a UNION ALL query counting rows from 11 tables. The 'Script Output' and 'Query Result' panes at the bottom show the execution results.

```
SELECT 'PATIENTS' AS table_name, COUNT(*) AS row_count FROM patients
UNION ALL
SELECT 'DOCTORS', COUNT(*) FROM doctors
UNION ALL
SELECT 'SPECIALIZATIONS', COUNT(*) FROM specializations
UNION ALL
SELECT 'DOCTOR_SPECIALIZATIONS', COUNT(*) FROM doctor_specializations
UNION ALL
SELECT 'APPOINTMENTS', COUNT(*) FROM appointments
UNION ALL
SELECT 'CONSULTATIONS', COUNT(*) FROM consultations
UNION ALL
SELECT 'MEDICAL_RESULTS', COUNT(*) FROM medical_results
UNION ALL
SELECT 'ROLES', COUNT(*) FROM roles
UNION ALL
SELECT 'DB_USERS', COUNT(*) FROM db_users
UNION ALL
SELECT 'PRIVILEGES', COUNT(*) FROM privileges
```

TABLE_NAME	ROW_COUNT
1 PATIENTS	0
2 DOCTORS	0
3 SPECIALIZATIONS	0
4 DOCTOR_SPECIALIZATIONS	0
5 APPOINTMENTS	0
6 CONSULTATIONS	0
7 MEDICAL_RESULTS	0
8 ROLES	0
9 DB_USERS	0
10 PRIVILEGES	0
11 ROLE_PRIVILEGES	0

Figura 12. Date de test inserate

1.6 Reguli de securitate aplicate asupra modelului

Proiectul aplică mai multe reguli de securitate asupra modelului medical, având în vedere faptul că baza de date conține informații personale și medicale sensibile. Regulile sunt organizate pe mai multe direcții: protecția datelor, auditarea activităților, gestiunea utilizatorilor, controlul accesului și securizarea operațiilor realizate la nivelul bazei de date.

Reguli de securitate aplicate:

Layer 1: Criptarea datelor

- Datele sensibile ale pacienților, precum CNP-ul, diagnosticul, tratamentul și rezultatele medicale, vor fi protejate prin mecanisme de criptare.
- Criptarea este utilizată pentru a reduce riscul expunerii datelor confidențiale în cazul accesului neautorizat.

Layer 2: Auditarea activităților

- Operațiile importante asupra tabelelor medicale vor fi auditate.
- Vor fi urmărite acțiuni precum inserarea programărilor, modificarea consultațiilor și accesarea datelor medicale sensibile.
- Informațiile relevante vor fi salvate în tabelul AUDIT_LOG.

Layer 3: Gestiunea utilizatorilor și a resurselor

- Utilizatorii bazei de date vor fi creați în funcție de rolul lor în sistem.
- Fiecare categorie de utilizator va avea acces limitat la operațiile necesare activității sale.
- Vor fi utilizate profile și limite de resurse pentru controlul utilizării bazei de date.

Layer 4: Controlul accesului prin roluri și privilegii

- Accesul la date este organizat prin roluri precum RECEPTIONIST_ROLE, DOCTOR_ROLE, LAB_ROLE, AUDITOR_ROLE și ADMIN_ROLE.
- Fiecare rol primește privilegii diferite, în funcție de responsabilitățile utilizatorului.
- Se aplică principiul privilegiului minim, astfel încât utilizatorii să poată accesa doar datele și operațiile necesare.

Layer 5: Securitatea aplicațiilor și mascarea datelor

- Procedurile PL/SQL vor fi construite astfel încât să prevină atacurile de tip SQL Injection.
- Pentru utilizatorii cu acces limitat vor fi definite view-uri mascate.
- Datele personale și medicale vor fi afișate parțial acolo unde accesul complet nu este necesar.

2. Criptarea Datelor

Pentru protejarea datelor sensibile din modelul medical, a fost implementat un mecanism de criptare folosind pachetul DBMS_CRYPTO. Au fost vizate informații precum CNP-ul pacientului, diagnosticul, tratamentul, rezultatele medicale și observațiile confidențiale.

2.1 Date sensibile în modelul medical

Modelul medical conține date cu caracter personal și date referitoare la starea de sănătate a pacienților. Aceste informații au un nivel ridicat de confidențialitate, deoarece pot identifica direct o persoană sau pot descrie detalii medicale asociate acesteia.

În această categorie intră atribute precum CNP-ul, datele de contact, diagnosticul, tratamentul, rezultatele medicale și observațiile confidențiale.

2.2 Implementarea criptării

Pentru implementarea criptării a fost utilizat pachetul Oracle DBMS_CRYPTO. În tabelele care conțin date sensibile au fost adăugate coloane suplimentare de tip RAW, în care sunt salvate valorile criptate.

Criptarea a fost aplicată asupra următoarelor atribute:

PATIENTS.cnp

CONSULTATIONS.diagnosis

CONSULTATIONS.treatment

MEDICAL_RESULTS.result_value

MEDICAL_RESULTS.confidential_notes

Pentru realizarea operațiilor de criptare și decriptare a fost creat pachetul PL/SQL `medical_crypto_pkg`, care conține două funcții:

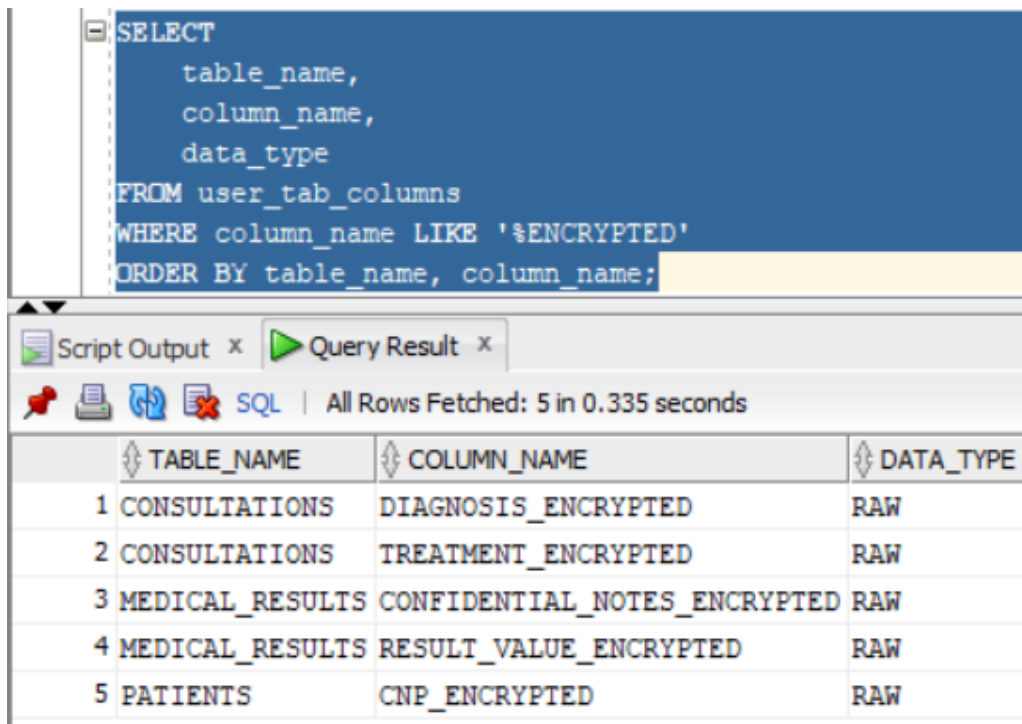
`encrypt_value` – transformă o valoare text într-o valoare criptată;

`decrypt_value` – transformă valoarea criptată înapoi în textul inițial.

Algoritmul folosit este AES256, iar valorile criptate sunt stocate separat de valorile inițiale, pentru a putea demonstra diferența dintre datele în clar și datele protejate.

Pentru stocarea valorilor criptate au fost adăugate coloane suplimentare de tip RAW în tabelele care conțin date sensibile.

```
ALTER TABLE patients ADD (  
    cnp_encrypted RAW(2000)  
);  
  
ALTER TABLE consultations ADD (  
    diagnosis_encrypted RAW(2000),  
    treatment_encrypted RAW(2000)  
);  
  
ALTER TABLE medical_results ADD (  
    result_value_encrypted RAW(2000),  
    confidential_notes_encrypted RAW(2000)  
);
```



The screenshot shows a SQL query execution window. The query is as follows:

```
SELECT
    table_name,
    column_name,
    data_type
FROM user_tab_columns
WHERE column_name LIKE '%ENCRYPTED'
ORDER BY table_name, column_name;
```

The results are displayed in a table with the following structure:

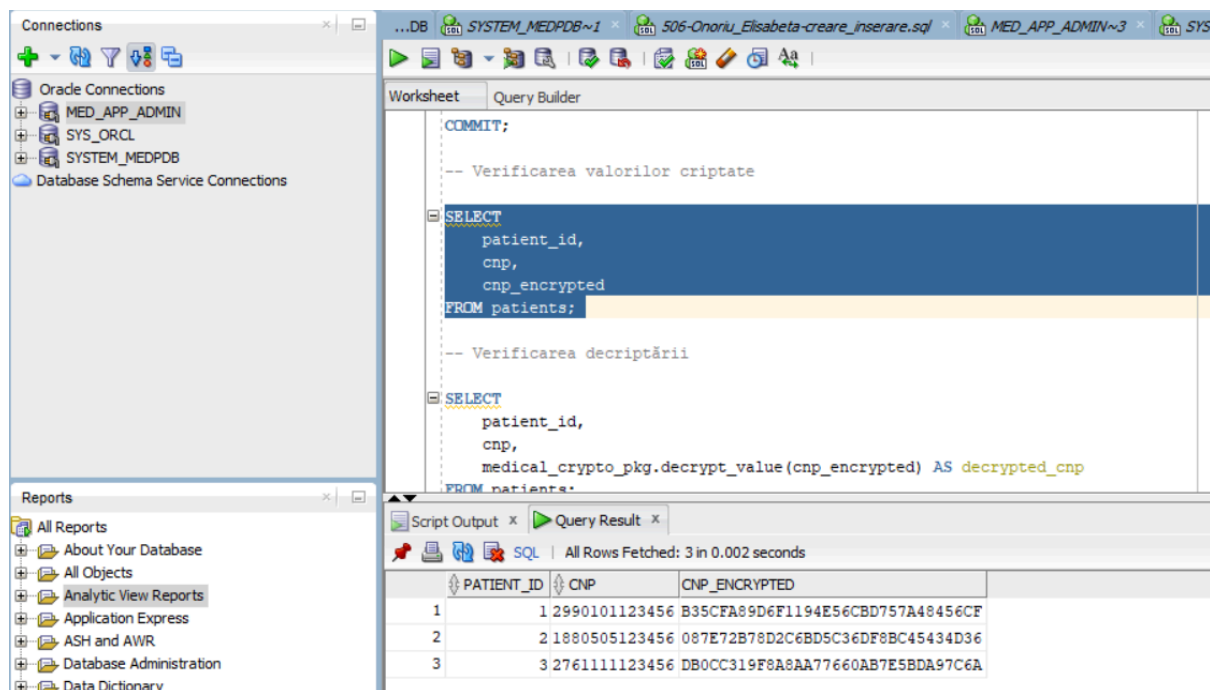
	TABLE_NAME	COLUMN_NAME	DATA_TYPE
1	CONSULTATIONS	DIAGNOSIS_ENCRYPTED	RAW
2	CONSULTATIONS	TREATMENT_ENCRYPTED	RAW
3	MEDICAL_RESULTS	CONFIDENTIAL_NOTES_ENCRYPTED	RAW
4	MEDICAL_RESULTS	RESULT_VALUE_ENCRYPTED	RAW
5	PATIENTS	CNP_ENCRYPTED	RAW

Figura 13. Verificarea coloanelor criptate adăugate în tabelele cu date sensibile

Rezultatul interogării confirmă existența coloanelor criptate în tabelele PATIENTS, CONSULTATIONS și MEDICAL_RESULTS.

2.3 Verificarea criptării

Pentru verificarea criptării, au fost interogate valorile originale și valorile criptate salvate în coloanele de tip RAW. Rezultatul arată că datele sensibile sunt stocate într-o formă neinteligibilă în coloanele criptate



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'MED_APP_ADMIN' connection selected. The main window is divided into a 'Worksheet' and a 'Query Builder' tab. The 'Worksheet' contains the following SQL script:

```
COMMIT;

-- Verificarea valorilor criptate

SELECT
  patient_id,
  cnp,
  cnp_encrypted
FROM patients;

-- Verificarea decriptării

SELECT
  patient_id,
  cnp,
  medical_crypto_pkg.decrypt_value(cnp_encrypted) AS decrypted_cnp
FROM patients;
```

Below the script, the 'Query Result' pane shows the output of the first query. It indicates 'All Rows Fetched: 3 in 0.002 seconds' and displays a table with three columns: PATIENT_ID, CNP, and CNP_ENCRYPTED. The data is as follows:

PATIENT_ID	CNP	CNP_ENCRYPTED
1	1 2990101123456	B35CFA89D6F1194E56CBD757A48456CF
2	2 1880505123456	087E72B78D2C6BD5C36DF8BC45434D36
3	3 276111123456	DB0CC319F8A8AA77660AB7E5BDA97C6A

Figura 14. Criptarea CNP-urilor pacientilor

The screenshot displays the Oracle SQL Developer environment. On the left, the 'Connections' pane shows 'MED_APP_ADMIN' as the selected connection. Below it, the 'Reports' pane lists various report categories. The main 'Worksheet' area contains the following SQL code:

```

COMMIT;

-- Verificarea valorilor criptate

SELECT
    patient_id,
    cnp,
    cnp_encrypted
FROM patients;

-- Verificarea decriptării

SELECT
    patient_id,
    cnp,
    medical_crypto_pkg.decrypt_value(cnp
FROM patients;

```

Below the SQL editor, the 'Query Result' tab shows the output of the second query. It indicates 'All Rows Fetched: 3 in 0.005 seconds' and displays a table with three columns: PATIENT_ID, CNP, and DECRYPTED_CNP.

PATIENT_ID	CNP	DECRYPTED_CNP
1	1 2990101123456	2990101123456
2	2 1880505123456	1880505123456
3	3 2761111123456	2761111123456

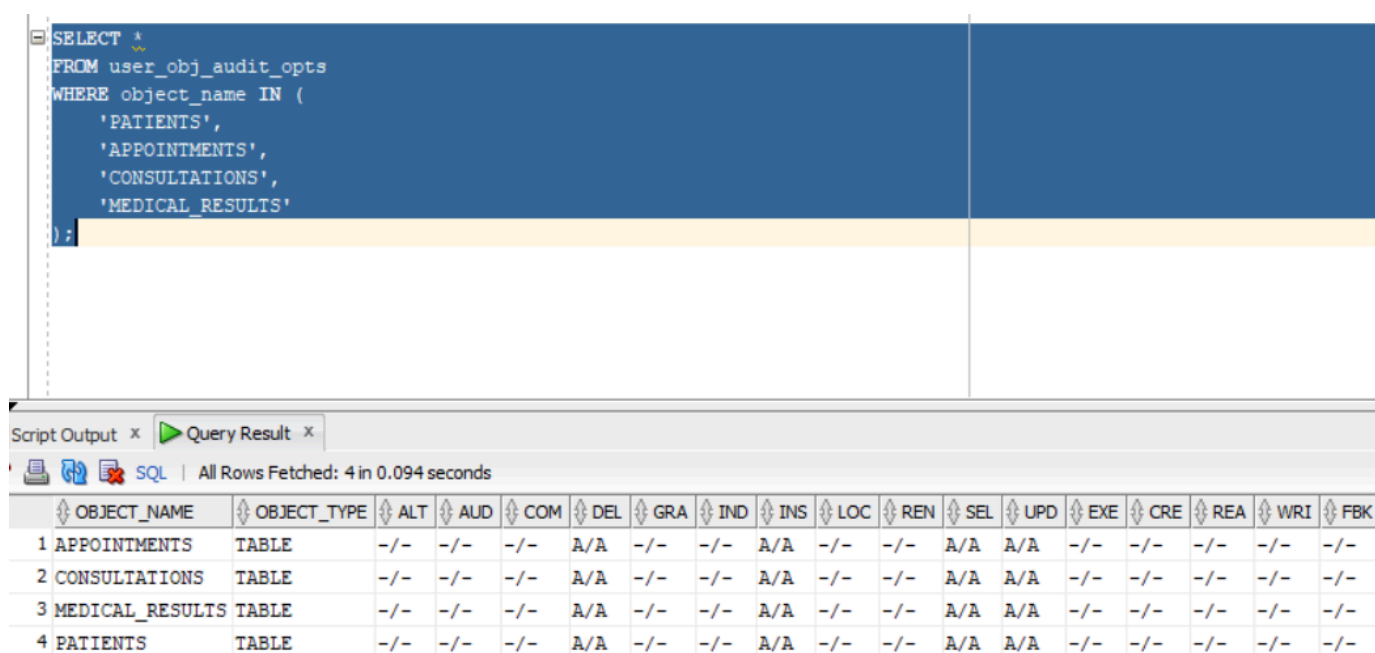
Figura 15. Decriptarea CNP-ului

3. Auditarea activităților asupra bazei de date

Auditarea este utilizată pentru monitorizarea operațiilor realizate asupra tabelelor importante din modelul medical. Prin mecanismele de audit se poate urmări cine a accesat sau modificat datele, ce operație a fost executată și asupra cărei tabele s-a realizat acțiunea.

3.1 Auditare standard

Pentru auditarea standard au fost activate operațiile SELECT, INSERT, UPDATE și DELETE asupra tabelelor care conțin date medicale și personale relevante: PATIENTS, APPOINTMENTS, CONSULTATIONS și MEDICAL_RESULTS. Configurația auditării a fost verificată prin interogarea view-ului USER_OBJ_AUDIT_OPTS.



```
SELECT *
FROM user_obj_audit_opts
WHERE object_name IN (
    'PATIENTS',
    'APPOINTMENTS',
    'CONSULTATIONS',
    'MEDICAL_RESULTS'
);
```

OBJECT_NAME	OBJECT_TYPE	ALT	AUD	COM	DEL	GRA	IND	INS	LOC	REN	SEL	UPD	EXE	CRE	REA	WRI	FBK
1 APPOINTMENTS	TABLE	-/-	-/-	-/-	A/A	-/-	-/-	A/A	-/-	-/-	A/A	A/A	-/-	-/-	-/-	-/-	-/-
2 CONSULTATIONS	TABLE	-/-	-/-	-/-	A/A	-/-	-/-	A/A	-/-	-/-	A/A	A/A	-/-	-/-	-/-	-/-	-/-
3 MEDICAL_RESULTS	TABLE	-/-	-/-	-/-	A/A	-/-	-/-	A/A	-/-	-/-	A/A	A/A	-/-	-/-	-/-	-/-	-/-
4 PATIENTS	TABLE	-/-	-/-	-/-	A/A	-/-	-/-	A/A	-/-	-/-	A/A	A/A	-/-	-/-	-/-	-/-	-/-

Figura 16. Verificarea auditării standard pentru tabelele medicale

3.2 Trigger-i de auditare

Pentru auditarea operațiilor DML au fost creați trigger-i care înregistrează automat acțiunile importante în tabela AUDIT_LOG. Aceștia sunt declanșați la efectuarea unor operații de tip INSERT, UPDATE sau DELETE asupra tabelelor relevante din modelul medical.

În cadrul proiectului au fost implementați trei trigger-i de auditare:

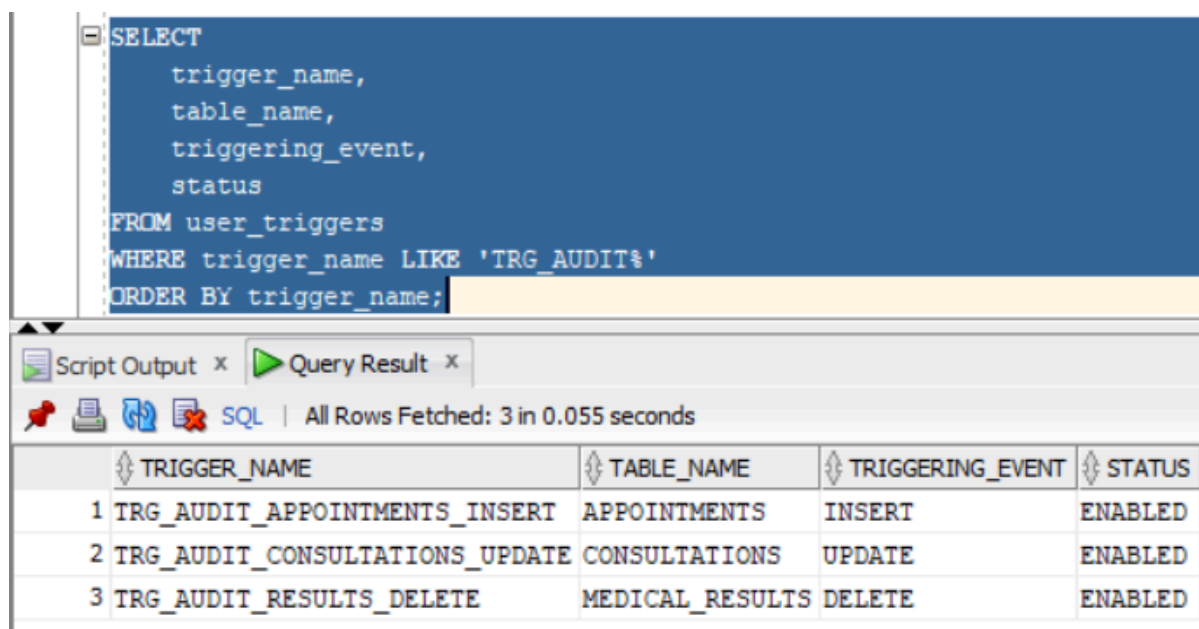
TRG_AUDIT_APPOINTMENTS_INSERT

TRG_AUDIT_CONSULTATIONS_UPDATE

TRG_AUDIT_RESULTS_DELETE

Primul trigger urmărește inserarea unei programări noi în tabela APPOINTMENTS, al doilea urmărește modificarea unei consultații în tabela CONSULTATIONS, iar al treilea urmărește ștergerea unui rezultat medical din tabela MEDICAL_RESULTS.

Pentru fiecare acțiune auditată sunt salvate informații precum utilizatorul care a executat operația, tipul acțiunii, tabela afectată, identificatorul înregistrării, valorile modificate și momentul execuției. Astfel, tabela AUDIT_LOG poate fi utilizată pentru urmărirea activităților relevante asupra datelor medicale.



```
SELECT
    trigger_name,
    table_name,
    triggering_event,
    status
FROM user_triggers
WHERE trigger_name LIKE 'TRG_AUDIT%'
ORDER BY trigger_name;
```

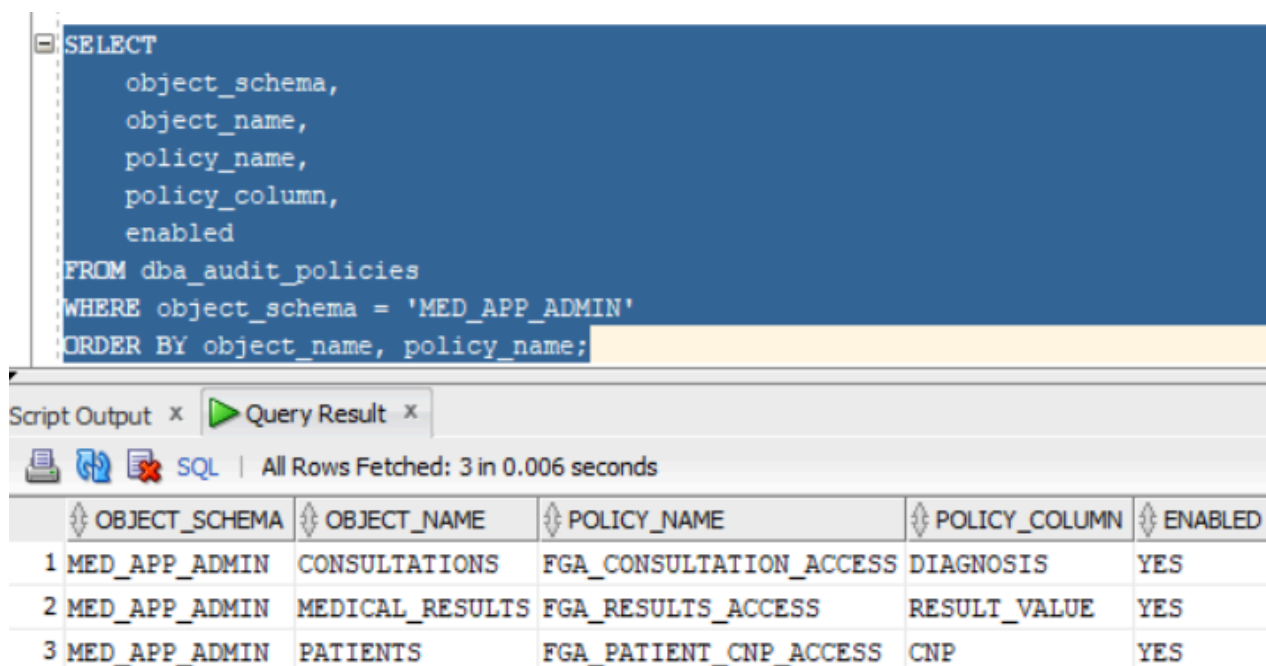
	TRIGGER_NAME	TABLE_NAME	TRIGGERING_EVENT	STATUS
1	TRG_AUDIT_APPOINTMENTS_INSERT	APPOINTMENTS	INSERT	ENABLED
2	TRG_AUDIT_CONSULTATIONS_UPDATE	CONSULTATIONS	UPDATE	ENABLED
3	TRG_AUDIT_RESULTS_DELETE	MEDICAL_RESULTS	DELETE	ENABLED

Figura 17. Verificarea trigger-ilor de auditare creați în baza de date

3.3 Politici de auditare

Pentru monitorizarea accesului la date sensibile au fost definite politici de auditare de tip Fine-Grained Auditing. Aceste politici urmăresc operațiile de citire asupra unor coloane importante din modelul medical, precum CNP-ul pacientului, diagnosticul, tratamentul, rezultatele medicale și observațiile confidențiale.

Spre deosebire de trigger-i, care urmăresc operații de modificare a datelor, politicile de auditare permit monitorizarea accesului prin SELECT asupra unor coloane sensibile.



```
SELECT
    object_schema,
    object_name,
    policy_name,
    policy_column,
    enabled
FROM dba_audit_policies
WHERE object_schema = 'MED_APP_ADMIN'
ORDER BY object_name, policy_name;
```

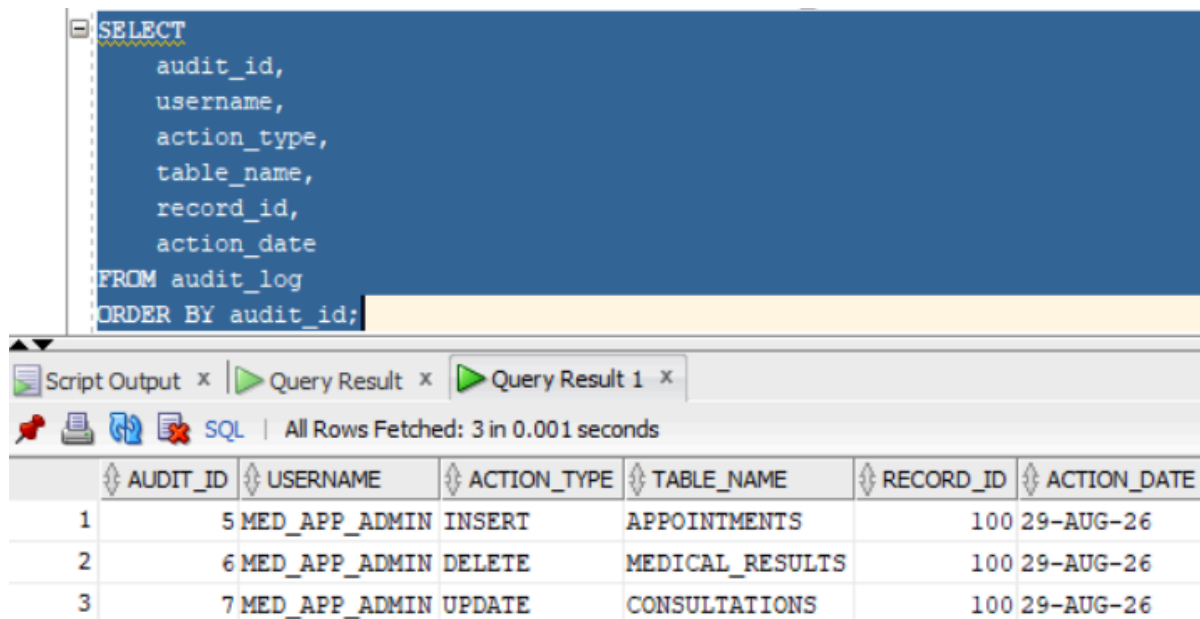
	OBJECT_SCHEMA	OBJECT_NAME	POLICY_NAME	POLICY_COLUMN	ENABLED
1	MED_APP_ADMIN	CONSULTATIONS	FGA_CONSULTATION_ACCESS	DIAGNOSIS	YES
2	MED_APP_ADMIN	MEDICAL_RESULTS	FGA_RESULTS_ACCESS	RESULT_VALUE	YES
3	MED_APP_ADMIN	PATIENTS	FGA_PATIENT_CNP_ACCESS	CNP	YES

Figura 18. Politici de auditare definite pentru coloane sensibile

3.4 Testarea auditării

Pentru testarea mecanismelor de auditare au fost executate operații de tip INSERT, UPDATE și DELETE asupra tabelelor monitorizate. Aceste operații au declanșat trigger-ii de auditare, iar rezultatele au fost salvate automat în tabela AUDIT_LOG.

Verificarea s-a realizat prin interogarea tabelii AUDIT_LOG, unde se pot observa utilizatorul care a executat operația, tipul acțiunii, tabela afectată, identificatorul înregistrării și momentul execuției.



```

SELECT
    audit_id,
    username,
    action_type,
    table_name,
    record_id,
    action_date
FROM audit_log
ORDER BY audit_id;

```

	AUDIT_ID	USERNAME	ACTION_TYPE	TABLE_NAME	RECORD_ID	ACTION_DATE
1	5	MED_APP_ADMIN	INSERT	APPOINTMENTS	100	29-AUG-26
2	6	MED_APP_ADMIN	DELETE	MEDICAL_RESULTS	100	29-AUG-26
3	7	MED_APP_ADMIN	UPDATE	CONSULTATIONS	100	29-AUG-26

Figura 19. Înregistrări generate în tabela AUDIT_LOG în urma testării mecanismelor de auditare

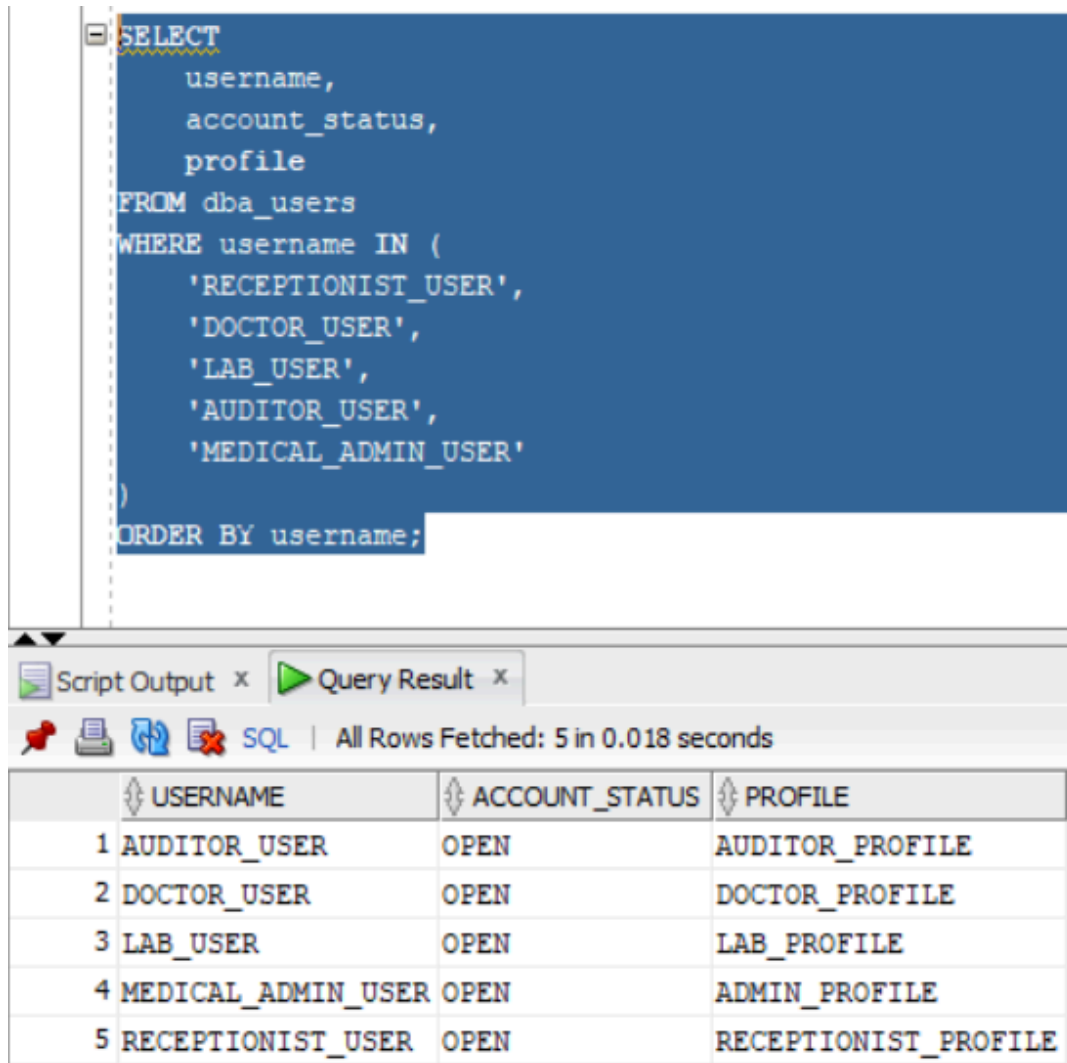
Rezultatul confirmă faptul că trigger-ii de auditare au fost declanșați în urma operațiilor de tip INSERT, UPDATE și DELETE. În tabela AUDIT_LOG au fost salvate informații despre utilizatorul care a executat operația, tipul acțiunii, tabela afectată, identificatorul înregistrării și data execuției.

4. Gestiunea utilizatorilor și a resurselor computaționale

Gestiunea utilizatorilor și a resurselor computaționale urmărește definirea conturilor Oracle utilizate în sistemul medical și aplicarea unor limite de resurse în funcție de rolul fiecărui utilizator. În cadrul proiectului au fost create conturi separate pentru recepție, medici, laborator, auditor și administrator, fiecare fiind asociat unui profil corespunzător.

4.1 Utilizatorii sistemului medical

Utilizatorii bazei de date au fost definiți în funcție de activitățile realizate în cadrul sistemului medical. Astfel, modelul include utilizatori pentru recepție, medici, laborator, audit și administrare. Separarea conturilor permite aplicarea ulterioară a unor reguli diferite de acces și limitarea resurselor utilizate.



The screenshot shows an Oracle SQL Developer interface. The top pane displays a SQL query that selects user information from the `dba_users` table, filtering for five specific usernames and ordering them alphabetically. The bottom pane shows the query results in a table format, with columns for `USERNAME`, `ACCOUNT_STATUS`, and `PROFILE`. The results show five users, all with an `OPEN` account status and a corresponding profile name.

```
SELECT
    username,
    account_status,
    profile
FROM dba_users
WHERE username IN (
    'RECEPTIONIST_USER',
    'DOCTOR_USER',
    'LAB_USER',
    'AUDITOR_USER',
    'MEDICAL_ADMIN_USER'
)
ORDER BY username;
```

	USERNAME	ACCOUNT_STATUS	PROFILE
1	AUDITOR_USER	OPEN	AUDITOR_PROFILE
2	DOCTOR_USER	OPEN	DOCTOR_PROFILE
3	LAB_USER	OPEN	LAB_PROFILE
4	MEDICAL_ADMIN_USER	OPEN	ADMIN_PROFILE
5	RECEPTIONIST_USER	OPEN	RECEPTIONIST_PROFILE

Figura 20. Utilizatori Oracle creați pentru sistemul medical

```

SELECT
    profile,
    resource_name,
    limit
FROM dba_profiles
WHERE profile IN (
    'RECEPTIONIST_PROFILE',
    'DOCTOR_PROFILE',
    'LAB_PROFILE',
    'AUDITOR_PROFILE',
    'ADMIN_PROFILE'
)
AND resource_name IN (
    'SESSIONS_PER_USER',
    'IDLE_TIME',
    'FAILED_LOGIN_ATTEMPTS'
)
ORDER BY profile, resource_name;

```

	PROFILE	RESOURCE_NAME	LIMIT
1	ADMIN_PROFILE	FAILED_LOGIN_ATTEMPTS	5
2	ADMIN_PROFILE	IDLE_TIME	60
3	ADMIN_PROFILE	SESSIONS_PER_USER	5
4	AUDITOR_PROFILE	FAILED_LOGIN_ATTEMPTS	3
5	AUDITOR_PROFILE	IDLE_TIME	40
6	AUDITOR_PROFILE	SESSIONS_PER_USER	1
7	DOCTOR_PROFILE	FAILED_LOGIN_ATTEMPTS	3
8	DOCTOR_PROFILE	IDLE_TIME	30
9	DOCTOR_PROFILE	SESSIONS_PER_USER	3
10	LAB_PROFILE	FAILED_LOGIN_ATTEMPTS	3
11	LAB_PROFILE	IDLE_TIME	20
12	LAB_PROFILE	SESSIONS_PER_USER	2
13	RECEPTIONIST_PROFILE	FAILED_LOGIN_ATTEMPTS	3
14	RECEPTIONIST_PROFILE	IDLE_TIME	20
15	RECEPTIONIST_PROFILE	SESSIONS_PER_USER	2

Figura 21. Profile de resurse definite pentru utilizatorii sistemului medical

4.2 Matrice proces-utilizator

Matricea proces-utilizator evidențiază ce procese pot fi realizate de fiecare categorie de utilizator. Această matrice ajută la stabilirea responsabilităților și la separarea operațiilor în funcție de rol.

Proces / Utilizator	Receptionist	Doctor	Laborator	Auditor	Administrator
Gestionare pacienți	Da	Citire	Nu	Citire	Da
Gestionare programări	Da	Citire	Nu	Citire	Da
Înregistrare consultații	Nu	Da	Nu	Citire	Da
Gestionare rezultate medicale	Nu	Citire	Da	Citire	Da
Vizualizare audit	Nu	Nu	Nu	Da	Da
Administrare utilizatori	Nu	Nu	Nu	Nu	Da
Administrare roluri și privilegii	Nu	Nu	Nu	Nu	Da

Matricea arată că utilizatorii obișnuiți au acces doar la procesele necesare activității lor, în timp ce administratorul are acces complet asupra sistemului. Auditorul are drepturi de consultare asupra datelor auditate, fără a modifica informațiile medicale.

4.3 Matrice entitate-proces

Matricea entitate-proces prezintă relația dintre procesele principale ale sistemului și entitățile bazei de date asupra cărora acestea acționează. Prin această matrice se poate observa ce tabele sunt implicate în fiecare operație funcțională.

Entitate / Proces	Gestionare pacienți	Gestionare programări	Înregistrare consultații	Gestionare rezultate	Auditare
PATIENTS	C, R, U	R	R	R	R
DOCTORS	R	R	R	R	R
SPECIALIZATIONS	R	R	R	R	R
APPOINTMENTS	R	C, R, U, D	R	Nu	R
CONSULTATIONS	R	R	C, R, U	R	R
MEDICAL_RESULTS	R	Nu	R	C, R, U, D	R
AUDIT_LOG	Nu	Nu	Nu	Nu	C, R

Legendă:

C = Create / inserare
R = Read / citire
U = Update / modificare
D = Delete / ștergere

Această matrice evidențiază operațiile permise asupra fiecărei entități. Tabelele care conțin date medicale sensibile, precum PATIENTS, CONSULTATIONS și MEDICAL_RESULTS, sunt accesate controlat, în funcție de procesul executat.

4.4 Matrice entitate-utilizator

Matricea entitate-utilizator stabilește nivelul de acces al fiecărei categorii de utilizator asupra entităților bazei de date. Aceasta reprezintă baza pentru acordarea ulterioară a privilegiilor și rolurilor Oracle.

Entitate / Utilizator	Receptionist	Doctor	Laborator	Auditor	Administrator
PATIENTS	C, R, U	R	R	R	C, R, U, D
DOCTORS	R	R	R	R	C, R, U, D
SPECIALIZATIONS	R	R	R	R	C, R, U, D
APPOINTMENTS	C, R, U, D	R	Nu	R	C, R, U, D
CONSULTATIONS	Nu	C, R, U	Nu	R	C, R, U, D
MEDICAL_RESULTS	Nu	R	C, R, U, D	R	C, R, U, D
AUDIT_LOG	Nu	Nu	Nu	R	C, R, U, D
DB_USERS	Nu	Nu	Nu	Nu	C, R, U, D
ROLES	Nu	Nu	Nu	Nu	C, R, U, D
PRIVILEGES	Nu	Nu	Nu	Nu	C, R, U, D

Matricea entitate-utilizator reflectă principiul privilegiului minim. De exemplu, recepționarul poate gestiona pacienții și programările, dar nu are acces la modificarea consultațiilor sau a rezultatelor medicale. Medicul poate lucra cu datele medicale ale pacienților, iar personalul de laborator poate gestiona rezultatele medicale. Auditorul are acces de citire asupra jurnalului de audit, iar administratorul are acces complet pentru administrarea sistemului.

4.5 Implementarea utilizatorilor, cotelor și profilelor

Pentru implementarea gestiunii utilizatorilor au fost create profile Oracle prin care se stabilesc limite de resurse pentru fiecare categorie de utilizator. Aceste limite includ numărul maxim de sesiuni active, timpul de inactivitate și numărul de încercări eșuate de autentificare.

Ulterior, au fost creați utilizatorii Oracle corespunzători rolurilor definite în sistem, fiecare fiind asociat cu profilul potrivit. Pentru fiecare utilizator a fost acordat dreptul minim de conectare la baza de date prin privilegiul CREATE SESSION.

```
SELECT
  profile,
  resource_name,
  limit
FROM dba_profiles
WHERE profile IN (
  'RECEPTIONIST_PROFILE',
  'DOCTOR_PROFILE',
  'LAB_PROFILE',
  'AUDITOR_PROFILE',
  'ADMIN_PROFILE'
)
AND resource_name IN (
  'SESSIONS_PER_USER',
  'IDLE_TIME',
  'FAILED_LOGIN_ATTEMPTS'
)
ORDER BY profile, resource_name;
```

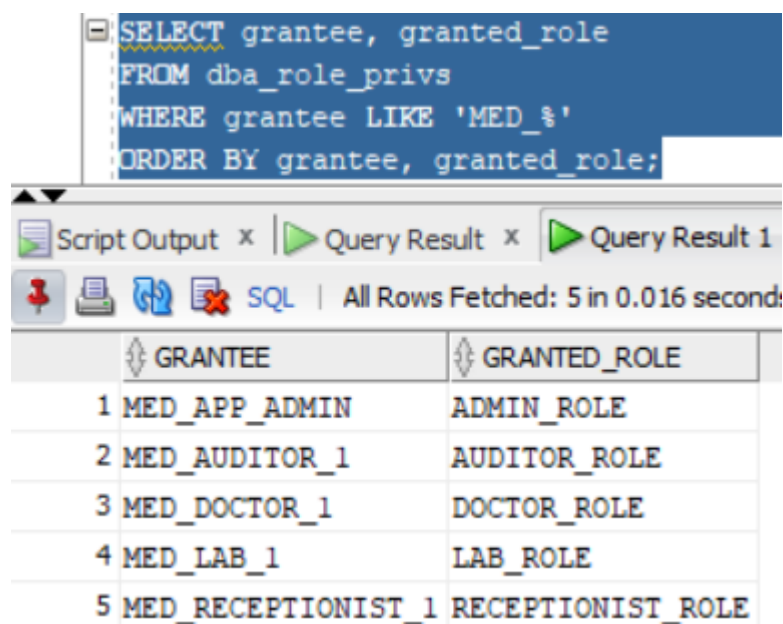
	PROFILE	RESOURCE_NAME	LIMIT
1	ADMIN_PROFILE	FAILED_LOGIN_ATTEMPTS	5
2	ADMIN_PROFILE	IDLE_TIME	60
3	ADMIN_PROFILE	SESSIONS_PER_USER	5
4	AUDITOR_PROFILE	FAILED_LOGIN_ATTEMPTS	3
5	AUDITOR_PROFILE	IDLE_TIME	40
6	AUDITOR_PROFILE	SESSIONS_PER_USER	1
7	DOCTOR_PROFILE	FAILED_LOGIN_ATTEMPTS	3
8	DOCTOR_PROFILE	IDLE_TIME	30
9	DOCTOR_PROFILE	SESSIONS_PER_USER	3
10	LAB_PROFILE	FAILED_LOGIN_ATTEMPTS	3
11	LAB_PROFILE	IDLE_TIME	20
12	LAB_PROFILE	SESSIONS_PER_USER	2
13	RECEPTIONIST_PROFILE	FAILED_LOGIN_ATTEMPTS	3
14	RECEPTIONIST_PROFILE	IDLE_TIME	20
15	RECEPTIONIST_PROFILE	SESSIONS_PER_USER	2

Figura 13. Profile de resurse definite pentru utilizatorii sistemului medical

Rezultatul interogării confirmă definirea profilelor Oracle asociate utilizatorilor sistemului medical. Pentru fiecare profil sunt stabilite limite privind numărul maxim de sesiuni active, timpul de inactivitate și numărul de încercări eșuate de autentificare.

5. Privilegii și roluri

În cadrul sistemului medical, accesul la date este controlat prin privilegiile și roluri. Fiecare utilizator primește doar drepturile necesare activității sale, conform principiului minimului privilegiu. În Oracle, privilegiile se acordă și se retrag prin comenzile GRANT și REVOKE, iar rolurile permit gruparea mai multor privilegii pentru o administrare mai simplă.



```

SELECT grantee, granted_role
FROM dba_role_privs
WHERE grantee LIKE 'MED_%'
ORDER BY grantee, granted_role;

```

	GRANTEE	GRANTED_ROLE
1	MED_APP_ADMIN	ADMIN_ROLE
2	MED_AUDITOR_1	AUDITOR_ROLE
3	MED_DOCTOR_1	DOCTOR_ROLE
4	MED_LAB_1	LAB_ROLE
5	MED_RECEPTIONIST_1	RECEPTIONIST_ROLE

Figura 22. Rezultatul interogării DBA_ROLE_PRIVS pentru rolurile acordate utilizatorilor MED

Rezultatul confirmă faptul că fiecare utilizator operațional a primit rolul corespunzător activității sale: recepționar, medic, laborator, auditor sau administrator.

5.1 Roluri definite în sistem

În cadrul sistemului medical au fost definite roluri distincte pentru fiecare categorie de utilizator. Rolurile permit gruparea privilegiilor și simplifică administrarea accesului la baza de date. Astfel, fiecare utilizator primește rolul corespunzător activității sale.

Au fost definite următoarele roluri:

RECEPTIONIST_ROLE – rol destinat recepționarului

DOCTOR_ROLE – rol destinat medicului

LAB_ROLE – rol destinat personalului de laborator

AUDITOR_ROLE – rol destinat auditorului

ADMIN_ROLE – rol destinat administratorului

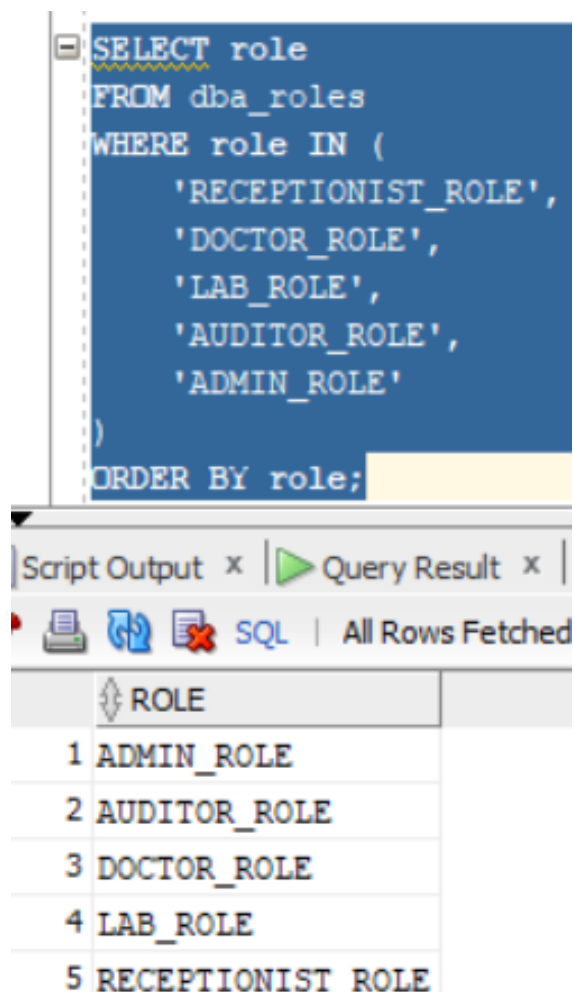


Figura 23: Rolurile definite în sistemul medical

5.2 Privilegii de sistem și privilegii pe obiecte

Privilegiile de sistem permit utilizatorilor să execute acțiuni generale în baza de date, cum ar fi conectarea la sistem. Privilegiile pe obiecte permit accesul la anumite tabele ale aplicației, prin operații precum SELECT, INSERT, UPDATE sau DELETE.

În cadrul proiectului, utilizatorii operaționali primesc doar drept de conectare, iar accesul la tabele este acordat prin roluri. Astfel, fiecare rol are doar privilegiile necesare activității sale.

```

SELECT grantee, owner, table_name, privilege
FROM dba_tab_privs
WHERE grantee IN (
    'RECEPTIONIST_ROLE',
    'DOCTOR_ROLE',

```

```

'LAB_ROLE',
'AUDITOR_ROLE',
'ADMIN_ROLE'
)
ORDER BY grantee, table_name, privilege;

```

	GRANTEE	OWNER	TABLE_NAME	PRIVILEGE
1	ADMIN_ROLE	MED_APP_ADMIN	APPOINTMENTS	DELETE
2	ADMIN_ROLE	MED_APP_ADMIN	APPOINTMENTS	INSERT
3	ADMIN_ROLE	MED_APP_ADMIN	APPOINTMENTS	SELECT
4	ADMIN_ROLE	MED_APP_ADMIN	APPOINTMENTS	UPDATE
5	ADMIN_ROLE	MED_APP_ADMIN	AUDIT_LOG	DELETE
6	ADMIN_ROLE	MED_APP_ADMIN	AUDIT_LOG	INSERT
7	ADMIN_ROLE	MED_APP_ADMIN	AUDIT_LOG	SELECT
8	ADMIN_ROLE	MED_APP_ADMIN	AUDIT_LOG	UPDATE
9	ADMIN_ROLE	MED_APP_ADMIN	CONSULTATIONS	DELETE
10	ADMIN_ROLE	MED_APP_ADMIN	CONSULTATIONS	INSERT
11	ADMIN_ROLE	MED_APP_ADMIN	CONSULTATIONS	SELECT
12	ADMIN_ROLE	MED_APP_ADMIN	CONSULTATIONS	UPDATE
13	ADMIN_ROLE	MED_APP_ADMIN	DOCTORS	DELETE
14	ADMIN_ROLE	MED_APP_ADMIN	DOCTORS	INSERT
15	ADMIN_ROLE	MED_APP_ADMIN	DOCTORS	SELECT
16	ADMIN_ROLE	MED_APP_ADMIN	DOCTORS	UPDATE
17	ADMIN_ROLE	MED_APP_ADMIN	MEDICAL_RESULTS	DELETE
18	ADMIN_ROLE	MED_APP_ADMIN	MEDICAL_RESULTS	INSERT
19	ADMIN_ROLE	MED_APP_ADMIN	MEDICAL_RESULTS	SELECT
20	ADMIN_ROLE	MED_APP_ADMIN	MEDICAL_RESULTS	UPDATE
21	ADMIN_ROLE	MED_APP_ADMIN	PATIENTS	DELETE
22	ADMIN_ROLE	MED_APP_ADMIN	PATIENTS	INSERT
23	ADMIN_ROLE	MED_APP_ADMIN	PATIENTS	SELECT
24	ADMIN_ROLE	MED_APP_ADMIN	PATIENTS	UPDATE
25	ADMIN_ROLE	MED_APP_ADMIN	SPECIALIZATIONS	DELETE
26	ADMIN_ROLE	MED_APP_ADMIN	SPECIALIZATIONS	INSERT

Figura 24. Privilegiile pe obiecte acordate rolurilor definite

Figura 16 prezintă privilegiile acordate rolurilor definite în sistemul medical. Fiecare rol primește acces doar la tabelele necesare activității sale, conform principiului minimului privilegiu.

5.3 Ierarhia rolurilor

În cadrul sistemului medical, rolurile operaționale sunt definite separat, în funcție de activitatea fiecărui utilizator. Pentru administrarea centralizată a sistemului, rolul ADMIN_ROLE primește rolurile principale ale aplicației. Astfel, administratorul poate avea acces la funcțiile de recepție, medic, laborator și audit.

Ierarhia rolurilor este următoarea:

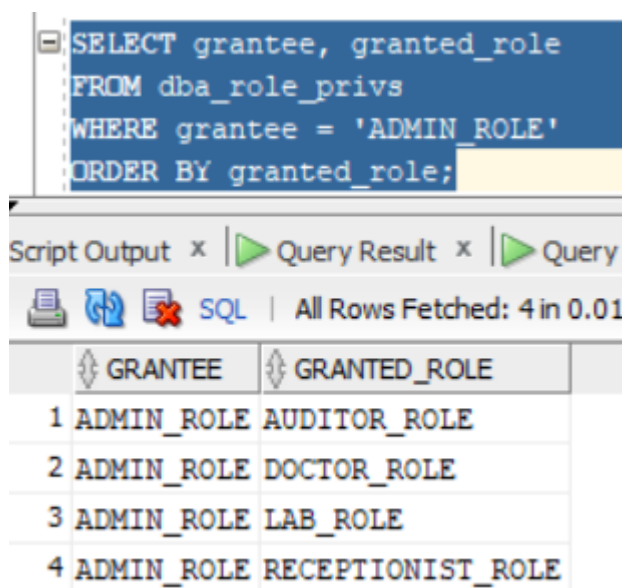
ADMIN_ROLE

└─ RECEPTIONIST_ROLE

└─ DOCTOR_ROLE

└─ LAB_ROLE

└─ AUDITOR_ROLE



The screenshot shows a SQL query window with the following query:

```
SELECT grantee, granted_role
FROM dba_role_privs
WHERE grantee = 'ADMIN_ROLE'
ORDER BY granted_role;
```

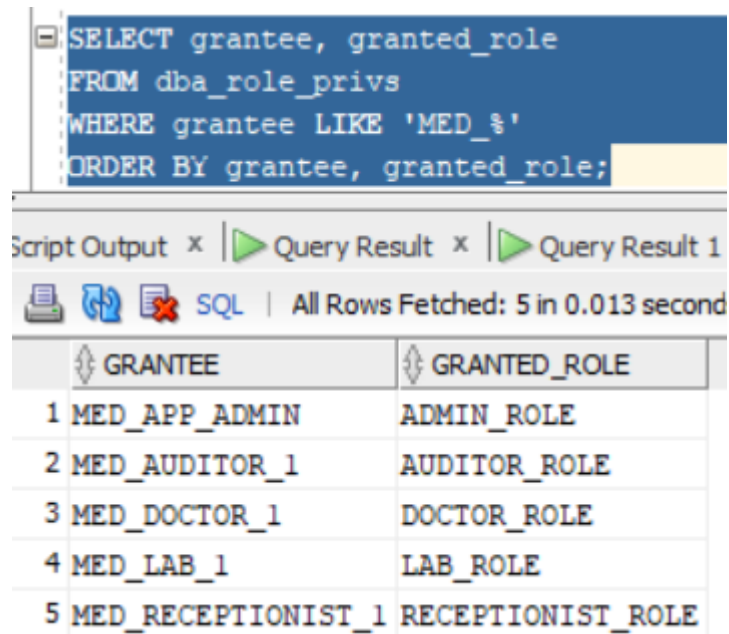
Below the query window, the results are displayed in a table with two columns: GRANTEE and GRANTED_ROLE. The results show that ADMIN_ROLE is granted four other roles: AUDITOR_ROLE, DOCTOR_ROLE, LAB_ROLE, and RECEPTIONIST_ROLE.

	GRANTEE	GRANTED_ROLE
1	ADMIN_ROLE	AUDITOR_ROLE
2	ADMIN_ROLE	DOCTOR_ROLE
3	ADMIN_ROLE	LAB_ROLE
4	ADMIN_ROLE	RECEPTIONIST_ROLE

Figura 25. Ierarhia rolurilor definite în sistemul medical

5.4 Testarea privilegiilor acordate

Testarea privilegiilor s-a realizat prin conectarea cu utilizatori diferiți și executarea unor operații permise și nepermise. Operațiile corespunzătoare rolului acordat au fost executate cu succes, iar operațiile neautorizate au generat erori de tip privilegii insuficiente.



The screenshot shows a SQL query window with the following text:

```
SELECT grantee, granted_role  
FROM dba_role_privs  
WHERE grantee LIKE 'MED_%'  
ORDER BY grantee, granted_role;
```

Below the query window, the 'Query Result' tab is active, displaying the results of the query. The status bar indicates 'All Rows Fetched: 5 in 0.013 second'.

	GRANTEE	GRANTED_ROLE
1	MED_APP_ADMIN	ADMIN_ROLE
2	MED_AUDITOR_1	AUDITOR_ROLE
3	MED_DOCTOR_1	DOCTOR_ROLE
4	MED_LAB_1	LAB_ROLE
5	MED_RECEPTIONIST_1	RECEPTIONIST_ROLE

Figura 26. Verificarea rolurilor acordate utilizatorilor sistemului medical

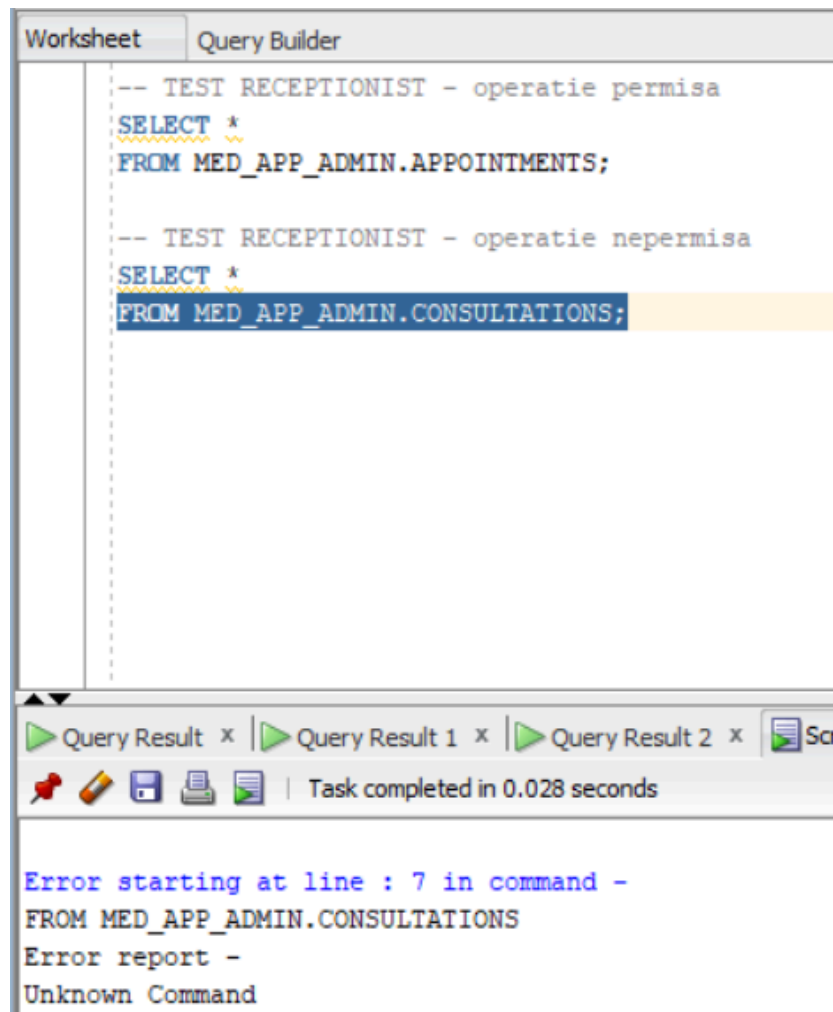


Figura 27. Testarea privilegiilor pentru utilizatorul MED_RECEPTIONIST_1

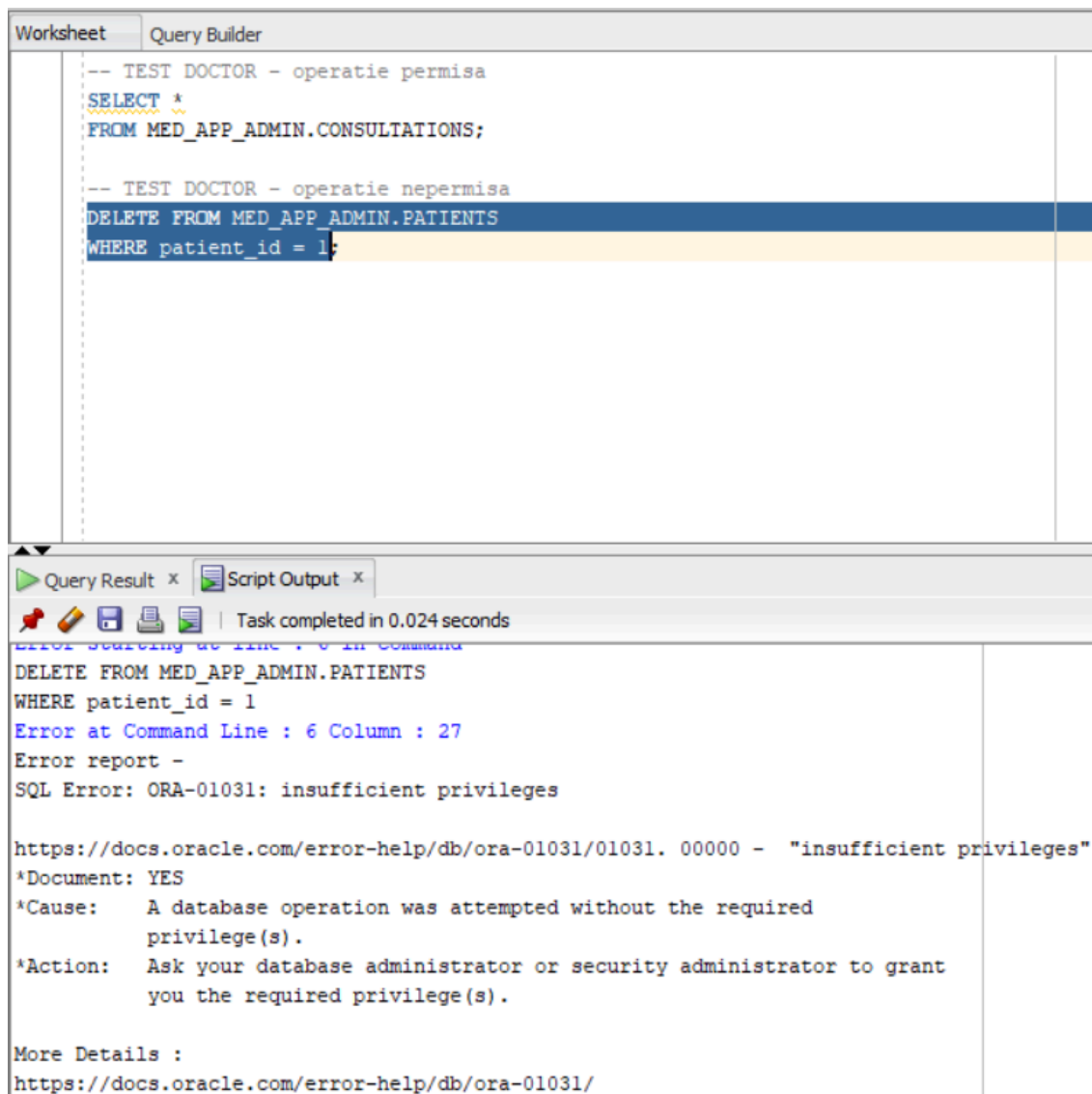


Figura 28. Testarea privilegiilor pentru utilizatorul MED_DOCTOR_1

6. Aplicațiile pe baza de date și securitatea datelor

În acest capitol este analizată securitatea aplicațiilor care interacționează cu baza de date. Configurarea utilizatorilor, rolurilor și privilegiilor nu este suficientă dacă procedurile aplicației sunt construite într-un mod vulnerabil. Din acest motiv, sunt tratate mecanisme prin care Oracle permite controlul suplimentar al accesului și prevenirea unor atacuri asupra datelor.

Capitolul urmărește utilizarea contextului aplicației pentru identificarea utilizatorului curent și pentru controlul operațiilor permise. De asemenea, este prezentat riscul de SQL Injection, prin compararea unei proceduri vulnerabile cu o procedură securizată. Scopul este de a evidenția diferența dintre o implementare nesigură, bazată pe concatenarea parametrilor în instrucțiuni SQL, și o implementare sigură, bazată pe validare și variabile de legătură.

6.1 Contextul aplicației

Contextul aplicației este un mecanism Oracle prin care pot fi stocate informații despre sesiunea curentă, cum ar fi utilizatorul aplicației sau rolul acestuia. Aceste informații pot fi folosite ulterior în proceduri, funcții sau politici de securitate pentru a controla accesul la date.

În cadrul proiectului, contextul aplicației este folosit pentru identificarea utilizatorului conectat în sistemul medical. Astfel, baza de date poate reține numele utilizatorului curent și îl poate utiliza în operații de verificare sau auditare.



```
BEGIN
    MED_APP_CONTEXT_PKG.set_current_user('MED_DOCTOR_1');
END;
/

SELECT SYS_CONTEXT('MED_APP_CTX', 'CURRENT_USER') AS utilizator_curent
FROM dual;
```

Script Output x Query Result x

SQL | All Rows Fetched: 1 in 0.001 seconds

UTILIZATOR_CURENT
1 MED_DOCTOR_1

Figura 29. Setarea și verificarea contextului aplicației

6.2 SQL Injection

SQL Injection reprezintă o vulnerabilitate care apare atunci când datele introduse de utilizator sunt concatenate direct într-o instrucțiune SQL. În acest caz, un atacator poate modifica logica interogării și poate obține acces neautorizat la informații din baza de date.

Într-un sistem medical, această vulnerabilitate este deosebit de periculoasă, deoarece poate permite accesul la date sensibile despre pacienți, consultații sau rezultate medicale. De exemplu, dacă numele unui pacient este introdus direct într-o interogare SQL fără validare, utilizatorul poate introduce caractere speciale sau condiții suplimentare pentru a returna mai multe înregistrări decât cele permise.

Pentru prevenirea atacurilor de tip SQL Injection, este recomandată evitarea concatenării directe a parametrilor în instrucțiunile SQL și utilizarea variabilelor de legătură. Acestea separă codul SQL de valorile introduse de utilizator și reduc riscul executării unor instrucțiuni neautorizate.

Exemplu conceptual de SQL Injection:

```
SELECT *  
FROM patients  
WHERE last_name = 'Popescu';
```

O interogare construită nesigur poate deveni vulnerabilă dacă utilizatorul introduce o valoare de forma:

```
' OR '1'='1
```

În acest caz, condiția devine mereu adevărată și pot fi afișate mai multe date decât ar trebui.

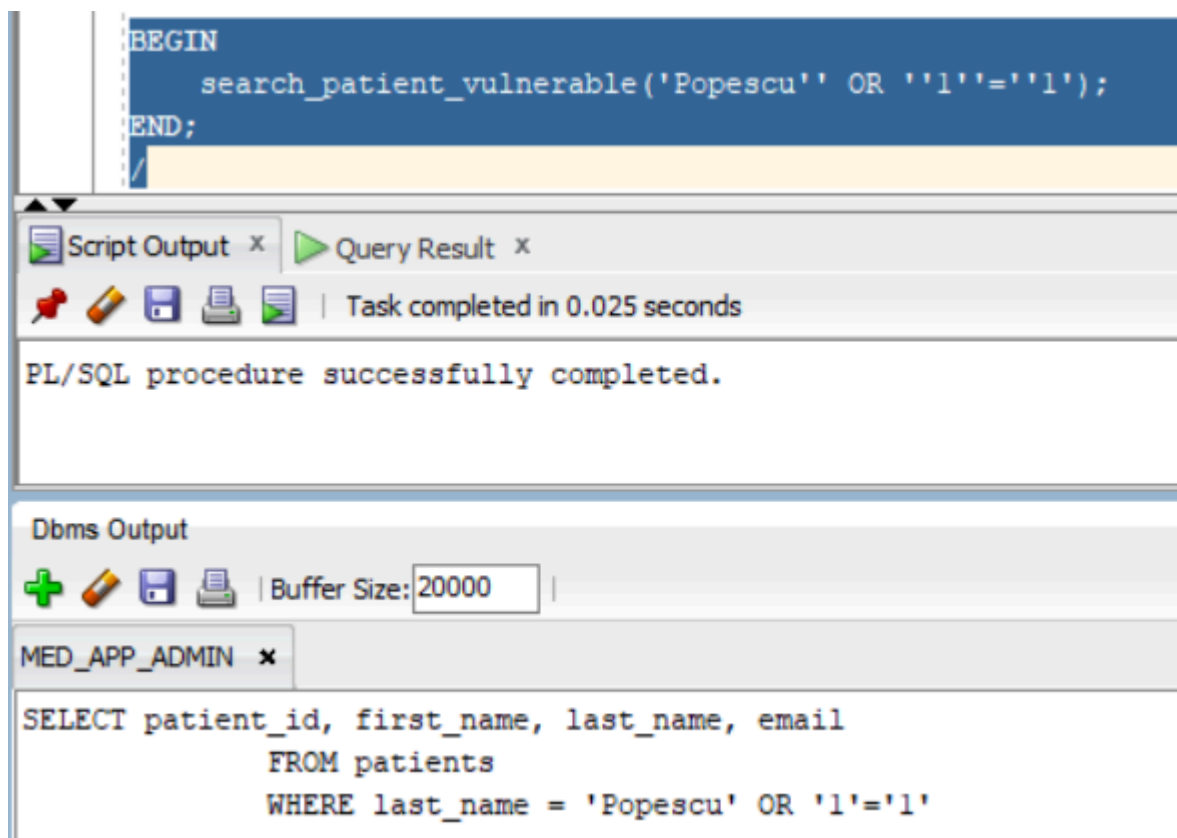
Pentru prevenirea SQL Injection, în proiect au fost aplicate următoarele măsuri:

- utilizarea procedurilor PL/SQL pentru accesul controlat la date;
- folosirea parametrilor în locul concatenării directe a valorilor în SQL;
- utilizarea variabilelor de legătură în procedura securizată;
- acordarea accesului prin roluri și privilegii;
- identificarea utilizatorului curent prin contextul aplicației;
- auditarea operațiilor asupra datelor sensibile.

6.3 Procedură vulnerabilă

Pentru evidențierea riscului de SQL Injection, a fost creată o procedură vulnerabilă care caută pacienți după numele de familie. Procedura construiește instrucțiunea SQL prin concatenarea directă a parametrului primit, ceea ce permite modificarea logicii interogării de către utilizator.

Această abordare este nesigură deoarece valoarea introdusă nu este tratată strict ca dată, ci poate deveni parte din instrucțiunea SQL executată.



The screenshot displays a database IDE interface. At the top, a SQL script is shown in a blue editor window:

```
BEGIN
  search_patient_vulnerable('Popescu' OR '1'='1');
END;
```

Below the editor, a status bar indicates "Task completed in 0.025 seconds". A "Script Output" window shows the message: "PL/SQL procedure successfully completed." At the bottom, the "Dbms Output" window is open, showing the SQL query executed:

```
SELECT patient_id, first_name, last_name, email
  FROM patients
 WHERE last_name = 'Popescu' OR '1'='1'
```

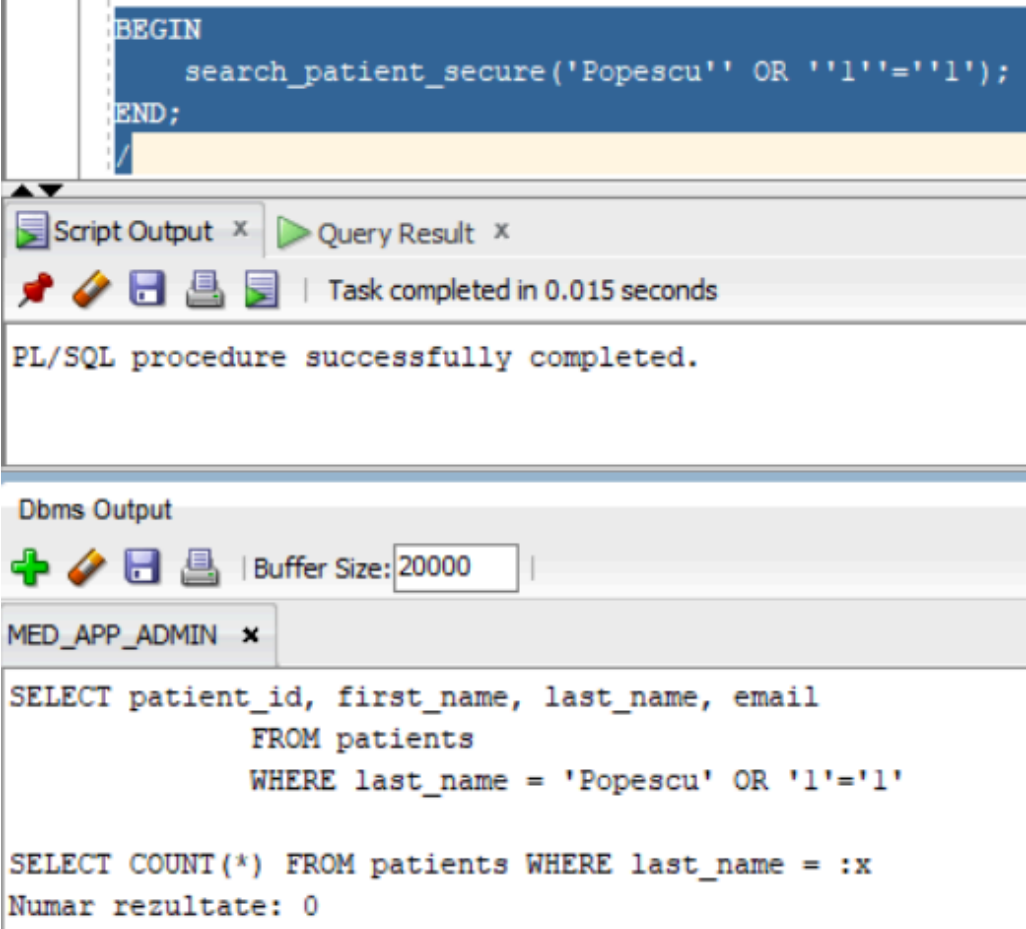
Figura 30. Testarea procedurii vulnerabile la SQL Injection

Figura 30 evidențiază faptul că procedura vulnerabilă construiește instrucțiunea SQL prin concatenarea directă a parametrului introdus. Valoarea OR '1'='1' modifică logica interogării și poate permite afișarea tuturor înregistrărilor din tabelul pacienților.

6.4 Procedură securizată

Pentru a preveni atacurile de tip SQL Injection, a fost creată o procedură securizată care utilizează o variabilă de legătură în locul concatenării directe a parametrului în instrucțiunea SQL. În acest mod, valoarea introdusă de utilizator este tratată ca dată, nu ca parte a comenzii SQL.

Prin această abordare, logica interogării nu mai poate fi modificată de inputul utilizatorului, iar riscul de acces neautorizat la date este redus.



```
BEGIN
    search_patient_secure('Popescu' OR '1'='1');
END;
```

Script Output x Query Result x

Task completed in 0.015 seconds

PL/SQL procedure successfully completed.

Dbms Output

Buffer Size: 20000

MED_APP_ADMIN x

```
SELECT patient_id, first_name, last_name, email
      FROM patients
     WHERE last_name = 'Popescu' OR '1'='1'

SELECT COUNT(*) FROM patients WHERE last_name = :x
Numar rezultate: 0
```

Figura 31. Testarea procedurii securizate împotriva SQL Injection

Figura 31 evidențiază faptul că procedura securizată utilizează o variabilă de legătură, astfel încât valoarea introdusă este tratată strict ca parametru. În consecință, încercarea de modificare a logicii interogării nu mai produce efectul specific unui atac de tip SQL Injection.

7. Mascarea datelor

Mascarea datelor este o tehnică de securitate utilizată pentru protejarea informațiilor sensibile. Într-un sistem medical, anumite date precum CNP-ul, adresa, telefonul, emailul sau diagnosticul pacientului nu trebuie afișate integral tuturor utilizatorilor.

În cadrul proiectului, mascarea datelor este aplicată asupra tabelului PATIENTS și asupra informațiilor medicale sensibile. Scopul este ca utilizatorii să poată consulta anumite informații necesare activității lor, fără a avea acces complet la date personale sau medicale confidențiale.

Pentru implementare au fost definite funcții de mascare, care transformă valorile sensibile într-o formă parțial ascunsă. Aceste funcții sunt apoi utilizate în view-uri mascate, astfel încât accesul la date să fie controlat fără modificarea informațiilor originale din tabele.

7.1 Date mascate în modelul medical

În modelul medical, datele care trebuie mascate sunt cele care pot identifica pacientul sau pot dezvălui informații medicale confidențiale.

Principalele date mascate sunt:

Tabel	Date mascate
PATIENTS	CNP, EMAIL, PHONE, ADDRESS
CONSULTATIONS	DIAGNOSIS, TREATMENT
MEDICAL_RESULTS	RESULT_VALUE, CONFIDENTIAL_NOTES

Aceste date sunt protejate deoarece includ informații personale și medicale sensibile. Mascarea permite afișarea parțială a informațiilor, fără expunerea completă a datelor originale.

7.2 Funcții de mascare

Pentru protejarea datelor sensibile au fost create funcții de mascare. Acestea afișează doar o parte din valoarea originală și înlocuiesc restul caracterelor cu simbolul *.

Funcțiile sunt folosite ulterior în view-uri mascate, astfel încât datele originale să rămână nemodificate în tabele.

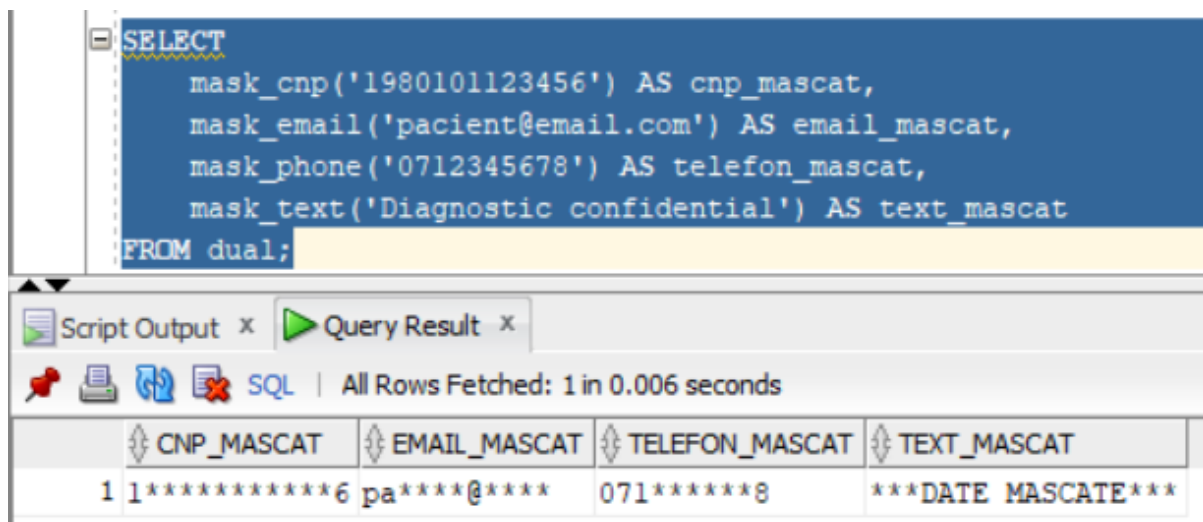


Figura 32. Testarea funcțiilor de mascare a datelor

Figura 32 prezintă rezultatul funcțiilor de mascare aplicate asupra unor date personale și medicale. Valorile originale sunt ascunse parțial sau complet, fără a modifica datele din tabele.

7.3 View-uri mascate

View-urile mascate sunt utilizate pentru afișarea controlată a datelor sensibile. În loc ca utilizatorii să acceseze direct tabelele originale, aceștia pot primi acces la view-uri în care anumite câmpuri sunt ascunse parțial sau complet.

În proiect, au fost create view-uri pentru pacienți și rezultate medicale, folosind funcțiile de mascare definite anterior.

SELECT *									
FROM v_patients_masked;									
Script Output x Query Result x									
All Rows Fetched: 3 in 0.011 seconds									
PATIENT_ID	FIRST_NAME	LAST_NAME	CNP_MASKED	EMAIL_MASKED	PHONE_MASKED	ADDRESS_MASKED	DATE_OF_BIRTH	CREATED_DATE	
1	Ana	Marin	2*****6	an****@****	073*****1	***DATE MASCATE***	01-JAN-99	29-AUG-26	
2	Mihai	Stan	1*****6	mi****@****	073*****2	***DATE MASCATE***	05-MAY-88	29-AUG-26	
3	Elena	Georgescu	2*****6	el****@****	073*****3	***DATE MASCATE***	11-NOV-76	29-AUG-26	

Figura 33a. View-uri mascate pentru protejarea datelor sensibile

SELECT *						
FROM v_medical_results_masked;						
Script Output x Query Result x						
All Rows Fetched: 1 in 0.002 seconds						
RESULT_ID	PATIENT_ID	DOCTOR_ID	RESULT_TYPE	RESULT_VALUE_MASKED	RESULT_DATE	CONFIDENTIAL_NOTES_MASKED
1	1	1	Analiza sange	***DATE MASCATE***	29-AUG-26	***DATE MASCATE***

Figura 33b. View-uri mascate pentru protejarea datelor sensibile

7.4 Testarea mascării

Testarea mascării s-a realizat prin compararea datelor originale din tabelul PATIENTS cu datele afișate prin view-ul V_PATIENTS_MASKED. Rezultatul demonstrează că informațiile sensibile sunt ascunse în view, fără ca datele originale din tabel să fie modificate.

SELECT patient id, first_name, last_name, cnp, email, phone, address						
FROM patients;						
Script Output x Query Result x						
All Rows Fetched: 3 in 0.005 seconds						
PATIENT_ID	FIRST_NAME	LAST_NAME	CNP	EMAIL	PHONE	ADDRESS
1	Ana	Marin	2990101123456	ana.marin@email.com	0733000001	Bucuresti
2	Mihai	Stan	1880505123456	mihai.stan@email.com	0733000002	Ilfov
3	Elena	Georgescu	2761111123456	elena.georgescu@email.com	0733000003	Bucuresti

Figura 34. Datele originale din tabelul PATIENTS

PATIENT_ID	FIRST_NAME	LAST_NAME	CNP_MASKED	EMAIL_MASKED	PHONE_MASKED	ADDRESS_MASKED
1	Ana	Marin	2*****6	an****@****	073*****1	***DATE MASCATE***
2	Mihai	Stan	1*****6	mi****@****	073*****2	***DATE MASCATE***
3	Elena	Georgescu	2*****6	el****@****	073*****3	***DATE MASCATE***

Figura 35. Datele mascate afișate prin view-ul V_PATIENTS_MASKED

Figura 35 evidențiază aplicarea funcțiilor de mascare asupra datelor sensibile. CNP-ul, emailul, telefonul și adresa sunt afișate parțial sau complet ascuns, în timp ce valorile originale rămân păstrate în tabelul de bază.

Concluzii

Proiectul a avut ca scop proiectarea și securizarea unei baze de date Oracle pentru gestionarea programărilor medicale și a evidenței pacienților. Modelul realizat include entități specifice unui sistem medical, precum pacienți, medici, programări, consultații, rezultate medicale și utilizatori ai aplicației.

Pe parcursul proiectului au fost aplicate mai multe mecanisme de securitate, precum criptarea datelor sensibile, auditarea operațiilor asupra bazei de date, definirea utilizatorilor, rolurilor și privilegiilor, prevenirea SQL Injection și mascarea datelor personale și medicale.

Prin utilizarea rolurilor și privilegiilor, accesul la date a fost limitat în funcție de responsabilitățile fiecărui utilizator. De asemenea, prin proceduri securizate, context de aplicație și view-uri mascate, proiectul demonstrează metode prin care datele pacienților pot fi protejate împotriva accesului neautorizat.

În concluzie, baza de date proiectată respectă principiul minimului privilegiu și oferă un nivel suplimentar de protecție pentru informațiile medicale sensibile. Implementarea acestor mecanisme contribuie la creșterea confidențialității, integrității și trasabilității datelor din sistemul medical.